UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Appunti di
Sistemi Operativi II

Anno 2021

Valentino Bocchetti

Contents

1 Introduzione al corso **3**

1.1 Esame 3

2 Lezione del 08-03 **3**

2.1 I sistemi operativi 3

2.2 Sistemi mono e multi programmati 4

2.3 Sistemi paralleli 4

2.4 Sistemi distribuiti 4

2.5 Ambiente di elaborazione (Client-Server o Peer-to-Peer) 5

2.6 Cloud computing 5

2.7 Sistemi mobile 5

2.8 Avvio di un sistema operativo nel dettaglio 5

2.9 Tabella degli interrupt 6

2.10 Memoria centrale 6

2.11 Memoria cache 6

2.12 Memoria di massa 6

2.13 I processi 7

2.14 Activity 8

2.15 Politiche utilizzate dallo scheduler 9

2.16 Sistema mono e multi tasking 9

2.17 Throughput (utilizzo della CPU) 10

2.18 Overhead 10

3 Lezione del 12-03 **10**

3.1 Sincronizzazione dei processi 10

3.2 Comunicazione con i dispositivi 12

3.3 File system 12

3.4 La shell dei comandi 13

3.5 La memoria virtuale 13

3.6 Memoria virtuale in Linux 14

3.7 Spazio di indirizzi di un processo 15

4 Lezione del 15-03 **17**

4.1 Swapping 17

4.2 Memory Arbiter 17

4.3 La RAM nei sistemi UNIX 18

4.3.1 Risultati desiderati 18

4.3.2 Stack & heap 18

4.4 Memory Management Unit (MMU) 20

4.5 Segmentazione 20

4.6 Indirizzo logico 20

4.6.1 Segment selector 20

4.6.2 Offset 21

4.6.3 Descrittore di segmento 21

4.6.4 Traduzione degli indirizzi logici in indirizzi fisici 22

4.6.5 Privilegi di esecuzione 23

4.6.6 Tabelle dei Descrittori di Segmento 23

4.7 Segmentazione in sistemi LINUX . . . 24

4.7.1 I segmenti in Linux . . . 25

4.7.2 Spazio degli indirizzi lineari . . . 25

4.7.3 Pagina di indirizzi lineari . . . 25

4.7.4 Segmentazione vs Paginazione . . . 25

4.7.5 Page . . . 25

4.7.6 Tipi di Paginazione . . . 26

4.7.7 TLB . . . 27

5 Lezione del 19-03 **27**

5.1 La programmazione di sistema . . . 27

5.1.1 System call . . . 27

5.1.2 Fork . . . 28

5.1.3 Processo Zombie . . . 28

5.1.4 Esempio . . . 28

5.2 Gestione dei processi in Linux . . . 28

5.3 Process Descriptor . . . 29

5.4 Stati da un processo . . . 29

5.5 Identità di un processo . . . 29

5.6 Relazioni fra processi . . . 29

5.7 Le liste di processi RUNNING . . . 29

5.8 I file hash . . . 30

5.8.1 Funzione di accesso h . . . 30

5.8.2 Divisione . . . 30

5.8.3 Risoluzione delle Collisioni . . . 30
5.8.4 Tabella Hash per la runqueue . . . 30
5.9 Organizzazione dei processi . . . 31
5.9.1 Wait queue . . . 31
5.9.2 Struttura di una Wait queue . . . 31
5.10 Duplicazione delle risorse . . . 31
5.10.1 Chiamate di sistema in Linux . . . 32
5.11 Bomba fork . . . 32
5.12 Terminazione di un processo . . . 32
5.13 I thread . . . 33
5.13.1 Kernel-level e user-level thread . . . 33
5.13.2 Clone() . . . 34
5.13.3 Pthreads . . . 34

6 Lezione del 22-03 **34**
6.1 Kernel thread . . . 34
6.1.1 Processi . . . 35
6.1.2 Process switch . . . 35
6.2 Scheduling dei processi . . . 36
6.2.1 Scheduling policy . . . 36
6.2.2 Single e multi-core . . . 36
6.2.3 Tipologie di Scheduling . . . 37
6.2.4 Caratteristiche di uno scheduler . . . 37
6.2.5 Prelazione . . . 38

6.2.6 Algoritmo di scheduling . . . 38
6.2.7 Scheduling di processi convenzionali . . . 39
6.2.8 Quanto temporale di base . . . 39
6.2.9 Durata del quanto temporale di base . . . 39
6.2.10 Average sleep time . . . 40

7 Lezione del 26-03 **40**

7.1 Processi batch e interattivi . . . 40
7.2 Scheduling dei processi Real-time . . . 41
7.3 Strutture dati usati dallo scheduler . . . 41
7.3.1 Principali campi della struttura . . . 41
7.4 Scheduling dei processi nel dettaglio . . . 42
7.5 Funzioni utilizzate dallo scheduler . . . 43
7.6 Aggiornamento del time slice di processi Real-time . . . 43
7.7 Rendere un processo running . . . 44
7.7.1 Funzione schedule() . . . 44
7.7.2 Schedule() prima e dopo il context switch . . . 45
7.8 Gestione dei Page Frame . . . 46
7.9 Page Descriptor . . . 46
7.9.1 Campi di un Page Descriptor . . . 46
7.10 NUMA . . . 46
7.11 Zone di memoria . . . 47
7.12 Zoned Page Frame Allocator . . . 48
7.12.1 Per-CPU page frame cache . . . 49

7.13 Kernel Mapping di page frame . . . 49

7.14 Il problema della frammentazione esterna . . . 49

7.15 Il Buddy System . . . 49

7.15.1 Filosofia del Buddy System . . . 50

7.15.2 Strutture dati . . . 51

7.15.3 Liberazione di un blocco . . . 51

8 Lezione del 29-03 **51**

8.1 Per-CPU Page Frame Cache . . . 51

8.2 La funzione `__alloc_pages()` . . . 52

8.3 La funzione `__free_pages()` . . . 52

8.4 Memory Area Management . . . 53

8.5 Lo Slab Allocator nel dettaglio . . . 53

8.6 Cache Generiche e Specifiche . . . 54

8.7 Allocazione e Deallocazione di Slab . . . 56

8.8 Gli Object Descriptor . . . 56

8.9 Slab Coloring . . . 56

8.10 Local Cache e Memory Pool . . . 57

8.11 Processi utente e gestione della memoria . . . 57

8.12 Lo spazio degli indirizzi . . . 57

8.13 Regioni di memoria . . . 58

8.14 Strutture dati . . . 58

8.15 Ricerca di una regione di memoria . . . 59

8.15.1 Alberi red-black . . . 59

8.16 Linux e la ricerca di una regione di memoria 59

8.17 Diritti di accesso ad una regione di memoria 59

8.18 Demand Paging 60

8.19 Page Reclaiming 60

8.20 Selezione di una pagina target 60

8.20.1 Principi alla base di un PFRA 61

8.21 Periodic Reclaiming 62

9 Lezione del 31-03 **62**

9.1 Virtual File System (VFS) 62

9.1.1 Common File Model 63

9.1.2 Strutture dati del VFS 63

9.1.3 L'oggetto superblock 63

9.1.4 Operazioni sul superblock 64

9.1.5 L'oggetto inode 64

9.1.6 Il File Object 64

9.1.7 Dentry File 65

9.1.8 Tipi di filesystem 65

9.1.9 La gestione dei filesystem 65

9.1.10 I namespace 65

9.1.11 Filesystem Mounting 66

10 Lezione del 09-04 - I sistemi operativi Real-Time **66**

10.1 Event Driven 66

10.2 Time Driven 67

10.2.1 Problema 1 → Osservabilità degli eventi 67
10.2.2 Problema 2 → Ritardo di rilevazione 67
10.2.3 Problema 3 → Ordine di occorrenza 68
10.3 WatchDog 68
10.4 Task periodici e Aperiodici 68
10.5 Deadline 68
10.6 Scheduling Real-Time 68
10.7 Rate Monothonic Scheduler 68
10.7.1 Test di schedulabilità 69
10.8 EDF Scheduler 69
10.9 Sistemi Soft e Hard Real-Time 70
10.9.1 Gestione di task in contesti ibridi 70
10.9.2 Limiti dei comuni sistemi Real-Time 70
10.9.3 Desiderata per i Sistemi Real-Time 70
10.9.4 Prevedibilità 71
10.10 I linguaggi di programmazione 72
10.10.1 Real-Time Euclid 72
10.10.2 Real-Time Concurrent C 72
10.11 Sistemi Embedded 73
10.11.1 Caso Semplice I 73
10.11.2 Caso Semplice II 73
10.11.3 Caso Complesso I 73
10.11.4 Caso Complesso II 73
10.11.5 Arduino 74

10.11.6 Microcontroller 74

10.11.7 La scheda Arduino 74

10.11.8 Programmare in Arduini 74

10.12 RT Linux 74

10.13 RTAI 75

1 Introduzione al corso

`Ricevimento` (da concordare almeno 24 ore prima) → Venerdì 10-12 (chat privata su teams)

`Email` → silviobarra@unina.it con oggetto **[INFO-S02]** (iscritto al *anno di corso* e *corso di laurea* SO2 esame facoltativo o obbligatorio)

Obiettivi

- Scelte progettuali ed implementative di un reale sistema operativo a base linux
- Sistemi distribuiti e del cloud computing
- Sistemi operativi ad uso mobile
- IoT

Prove intercorso:

- Domande sul sistema linux
- Domande sui sistemi mobile

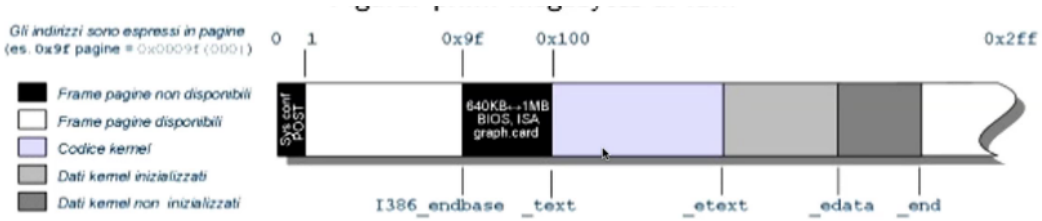
1.1 Esame

AGGIUNGI IMG

2 Lezione del 08-03

2.1 I sistemi operativi

Le componenti di un sistema di calcolo è così definita:



Un **sistema operativo** è difficile da definire (in base alle grandi variazioni che presenta). Una definizione è quella di definirlo come un insieme di software atti a gestire l'hardware sottostante.

Può essere visto come un allocatore di risorse:

- memoria;
- dispositiv di `I/O`;
- processi

Può essere visto come un programma di controllo:

- gestore dei programmi di sistema ed applicativi;
- verifica del corretto funzionamento dei dispositivo di `I/O`

2.2 Sistemi mono e multi programmati

Un sistema è detto **mono-programmato** quando in ogni momento è presente una sola **immagine** (pacchetto processo + dati)

Con l'introduzione del concetto di **spooling**, nascono i primi sistemi **multi-programmati** (il sistema è in grado di eseguire più job in concorrenza). Ciò comporta di tener conto della gestione delle risorse ad essi dedicate:

- Tempo di calcolo
- Memoria
- `I/O`

In questi sistemi si ha il aso di sistemi multi-utenti. Di conseguenza nascono i sistemi **time-sharing** (nascita dei primi PC). Ogni processo utilizza periodicamente un intervallo di tempo prestabilito (un **quanto**)

2.3 Sistemi paralleli

Si arriva alla nascita dei sistemi **Paralleli** dotati di più CPU :

- Symmetric multiprocessing (SMP) $\rightarrow$ ogni processore esegue una copia identica del SO
- Asymmetric multiprocessing (AMP) $\rightarrow$ a ciascun processore è assegnato un task specifico; il processore **master** schedua e allora il lavoro per i processori **slave**

2.4 Sistemi distribuiti

Ripartiscono l'elaborazione su diversi processori. Ciascun processore ha una memoria locale e i processori comunicano con gli altri attraverso di comunicazione quali bus ad alta velocità o rete. La rete può essere:

- Locale (LAN);
- Geografica (WAN);
- Metropolitana (MAN);
- Small (SAN).

2.5 Ambiente di elaborazione (Client-Server o Peer-to-Peer)

Esistono due principali tipi di server:

- server per l'elaborazione;
- file server

Nei sistemi Peer-to-Peer invece ciascun pc può essere server o client

- Il carico è suddiviso su più server (si evitano i sovraccarichi);
- Necessita una politica di gestione
- Nuova entità (**load balancer**)

2.6 Cloud computing

Paradigma di uso delle risorse computazionali attraverso il cloud (memorizzazione, archiviazione ed elaborazione dati)

2.7 Sistemi mobile

Il sistema operativo in questo caso ha orientamento specifico (a causa delle dimensioni del dispositivo)

2.8 Avvio di un sistema operativo nel dettaglio

All'avvio (o al reset) di un sistema il programma di **bootstrap** presente sul firmware inizializza i componenti del sistema, carica ed avvia il SO, che rimane in attesa di:

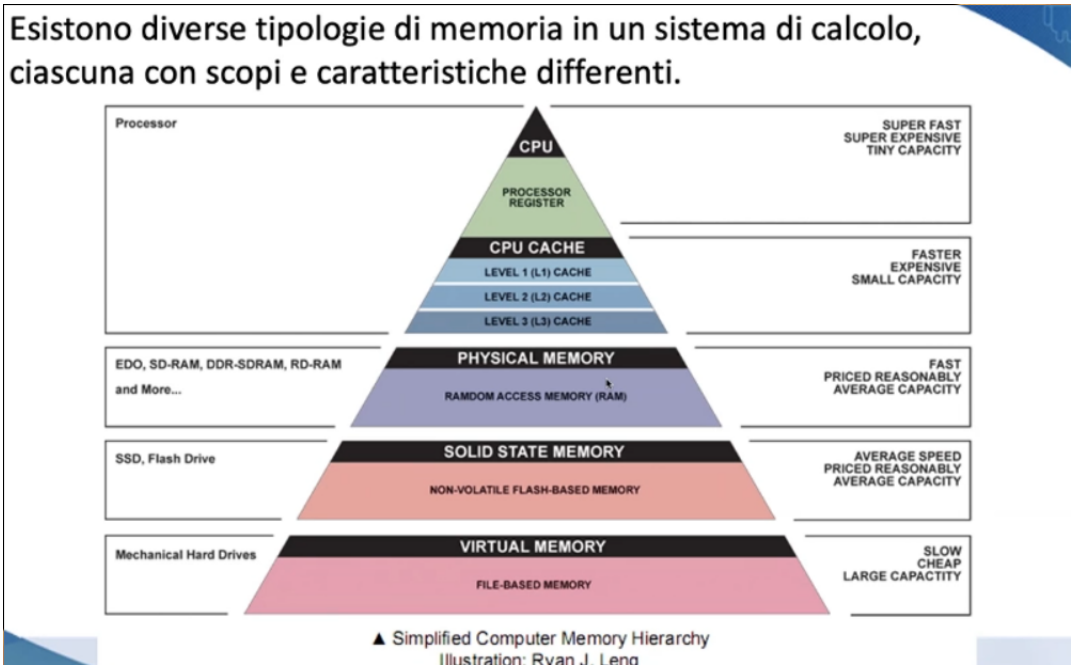
- Un programma applicativo da eseguire;
- Un interrupt → segnale generato da un controller di dispositivo o da un programma e invaito direttamente alla CPU, tramite bus;
- una exception → segnale generato da un programma a seguito di una condizione eccezionale di errore o di una specifica richiesta tramite system call.

La sequenza di avvio è:

Accensione → POST → controllo hard disk e Plug & Play → Avvio del bootloader → Avvio del SO

2.9 Tabella degli interrupt

Gli interrupt vengono gestiti mediante una strategia basata su una tabella



2.10 Memoria centrale

È rappresentata ccome un vettore di parole machina dalle centinaia di migliaia all'ordine dei miliardi di parole. Nella memoria centrale vengono caricati programmi e i dati

2.11 Memoria cache

Memoria molto veloce con lo scopo di ridurre al minimo gli accessi ad unità di memoria più lente. Bisogna garantire la coerenza delle informazioni memorizzate in una o più unità di memoria cache (il cui grado di complessità aumenta a seconda del tipo di sistema)

2.12 Memoria di massa

Divise in secondarie (dischi magnetici, solidi) e terziarie (dischi ottici, CD, DVD, nastri). Le responsabilità del SO rispetto alla gestione delle memorie di massa sono:

- gestione dello spazio libero;
- assegnazione dello spazio;
- scheduling del disco.

I dati vengono scritti nelle memoria mediante file

2.13 I processi

Un processo può essere definito come un programma di elaborazione o **job**. Un processo è un entità attiva, dotata di un program counter. La sua esecuzione è sequenziale e necessita di determinate risorse:

- CPU
- memoria
- dispositivi `I/O`;
- file

Il SO deve garantire alcune operazioni fondamentali per i processi:

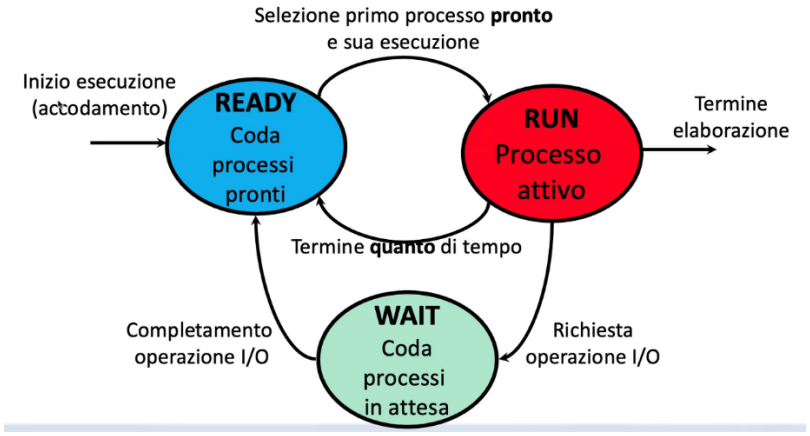
- creazione e cancellazione di processi;
- sospensione e ripristino di processi;
- sincronizzazione di processori;
- comunicazione fra processi;
- soluzioni per lo stato di **deadlock** di processi (questo compito spetta al kernel attraverso lo **scheduler**)

Gli stati di un processo

Ci si può trovare in 2 stati:

- attesa;
- pronto;

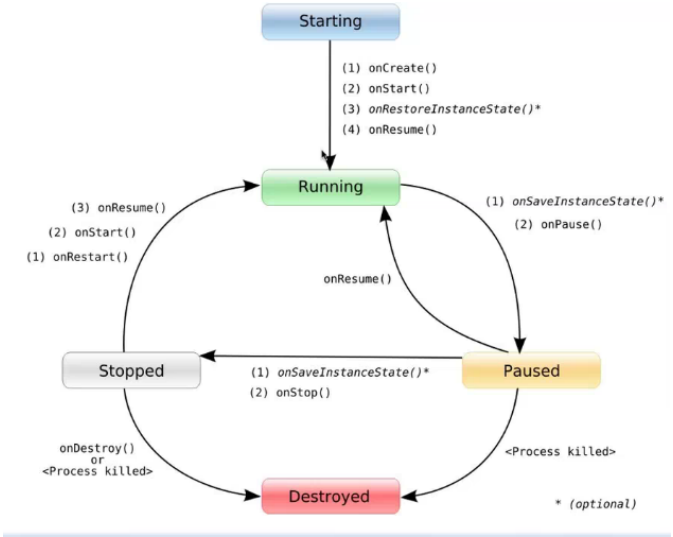
In ogni instante di tempo vi è un solo stato attivo



2.14 Activity

Una **Activity** è caratterizzata da un ciclo di vita ed in ogni istante si trova in uno dei seguenti stati:

- `Active/Resumed` (running) → essa è visibile e può ricevere input dall'utente;
- Paused → essa è parzialmente visibile a causa di un'altra activity in foreground, non può ricevere input dall'utente (*es* è in attesa a causa di un alert dialog presente al di sopra di essa);
- Stopped → essa non è visibile (resta in background)

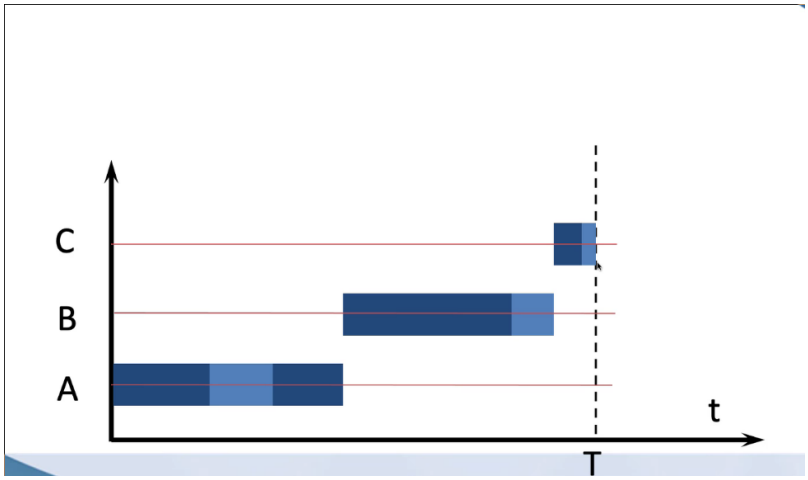


2.15 Politiche utilizzate dallo scheduler

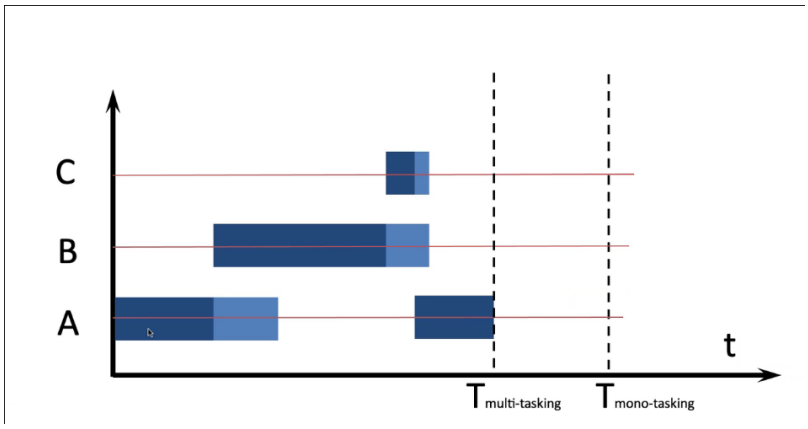
- **Preemtive** → la CPU in uso può essere tolta e passata a un altro in qualsiasi momento;
- **Non Preemtive** → una volta che il processo ha ottenuto l'us della CPU non può essere interrotta fino a che lui stesso non la rilascia;

2.16 Sistema mono e multi tasking

La gestione della CPU (mantenere in maniera efficiente il suo uso) è un obiettivo del sistema operativo. Nel caso di un sistema mono-tasking avremo:



Nel caso di un sistema multi-tasking avremo:



(*In blu scuro il tempo di utilizzo CPU e in chiaro il tempo di attesa di eventi esterni*)

2.17 Throughput (utilizzo della CPU)

Si ha dal rapporto di Tp/Tt dove:

- Tp = tempo dedicato dalla CPU alla esecuzione dei processi utente;
- Tt = tempo totale di osservazione

Si ha un'alta efficienza con un throughput tendente ad 1

2.18 Overhead

Tempo impiegato dal SO per trasferire il controllo da un programma ad un altro

3 Lezione del 12-03

3.1 Sincronizzazione dei processi

Uno dei compiti che spetta al SO è la sincronizzazione dei processi. Più processi su una macchina possono essere:

- In **competizione**
  - se si cerca di usare la stessa risorsa contemporaneamente;
- In **cooperazione**
  - quando uno ha bisogno dell'altro per evolvere;

La sincronizzazione avviene grazie a variabili condivise dette **semafori** o a scambio di messaggi tra processi

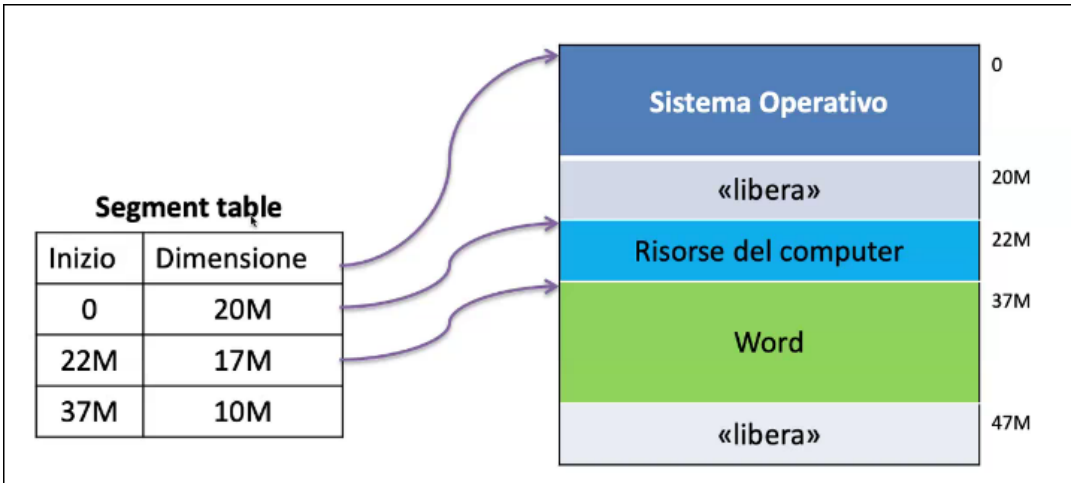
Ogni processo per essere eseguito deve sessere caricato in memoria. Nei sistemi multitasking permette di caricare più processi contemporaneamente.

Il gestore della memoria deve:

- Tenere traccia di quali parti della memoria sono usate e da chi;
- Decidere quali processi caricare quando diventa disponibile spazio in memoria;
- Allocare e deallocare memoria.

Per ogni blocco di memoria il SO mantiene un **descrittore** con le informazioni essenziali. La memoria fisica è suddivisa in un determinato numero di aeree (segmenti)

Ogni entry nella **segment table** contiene l'indirizzo di base di un segmento e la sua lunghezza



Lo svantaggio principale è la frammentazione (la memoria non viene sfruttata completamente). Per ovviare a ciò esiste un processo di compattamento → traslazione dei programmi.

Politiche di gestione dei frammenti

- **First-fit** → alloca il primo frammento libero sufficiente;
- **Best-fit** → alloca il più piccolo frammento libero sufficiente;
  - ricerca sull'intera lista e produce frammenti piccoli
- **Worst-fit** → alloca il più grande frammento
  - ricerca sull'intera lista e produce frammenti grandi

Nei moderni SO si utilizza la **paginazione dei segmenti**, ossia un segmento viene realizzato tramite un insieme di pagine

La suddivisione della memoria centrale in blocchi di dimensioni fisse (pagine) tipicamente 2KB o 4KB.

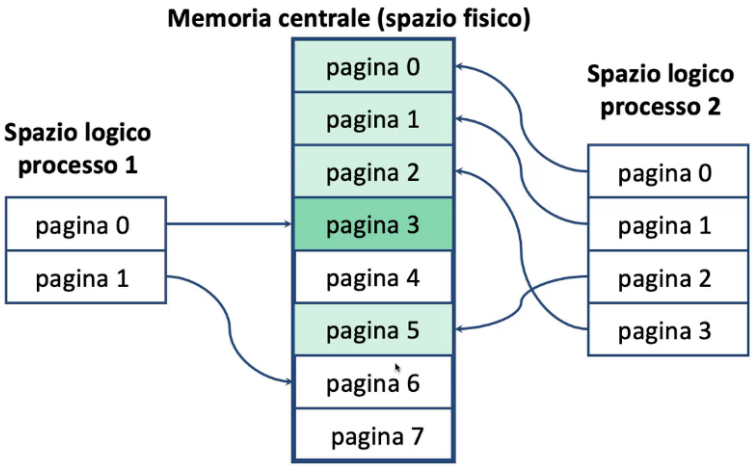
L'immagine di un processo viene suddivisa in pagine caricate all'interno della memoria principale in modo indipendente, anche in posizioni non contigue.

Il SO gestisce una **Page Table** che memorizza per ogni pagina (spazio logico) del processo e la sua posizione in memoria centrale (spazio fisico)

Frammentazione interna ed esterna

- **Frammentazione interna** → Con i blocchi di dimensioni fisse, all'interno di ogni blocco ci sarà uno spazio non utilizzato

- **Frammentazione esterna** → con i blocchi di dimensioni variabili, dopo un certo numero di allocazione e deallocazione di processi, si formeranno un certo numero di porzioni di memoria libere (buchi) di dimensioni insufficienti a contenere un processo



Se durante l'esecuzione del processo, la CPU fa riferimento ad un indirizzo che appartiene ad una pagina non caricata in memoria centrale (page fault), il SO provvede a mettere il processo in stato d'attesa e a far partire il caricamento della pagina richiesta (swap), dopodichè il processo può riprendere la sua esecuzione.

Nella paginazione esiste ancora il problema della frammentazione, che è però limitato al fatto che l'ultima pagina di ogni processo è solo parzialmente occupata. Mediamente si può considerare uno spreco pari a circa 1/2 pagina per ogni processo

3.2 Comunicazione con i dispositivi

Il gestore delle periferiche permette la comunicazione con le varie periferiche. Particolari programmi detti device driver gestiscono le operazioni di `in/out` tra le varie periferiche collegate alla macchina.

Le funzioni svolte dai device driver sono:

- Rendere trasparenti le caratteristiche fisiche tipiche di ogni dispositivo;
- Gestire la comunicazione dei segnali verso i dispositivi;
- Gestire i confiltti, nel caso in cui due o più task vogliono accedere contemporaneamente allo stesso dispositivo

3.3 File system

Gestisce le memorie di massa (organizza logicamente i dati e le possibili operazioni su di essi)

3.4 La shell dei comandi

L'interprete dei comandi o shell è quella parte più esterna di un SO che riceve ed elabora le istruzioni impartite da un utente Il SO presenta due tipi di intefaccia:

- testuale;
- grafica (diversa tra mobile e pc);

3.5 La memoria virtuale

Sistema `SW/HW` in grado di simulare uno spazio di memoria centrale maggiore di quello fisicamente presente

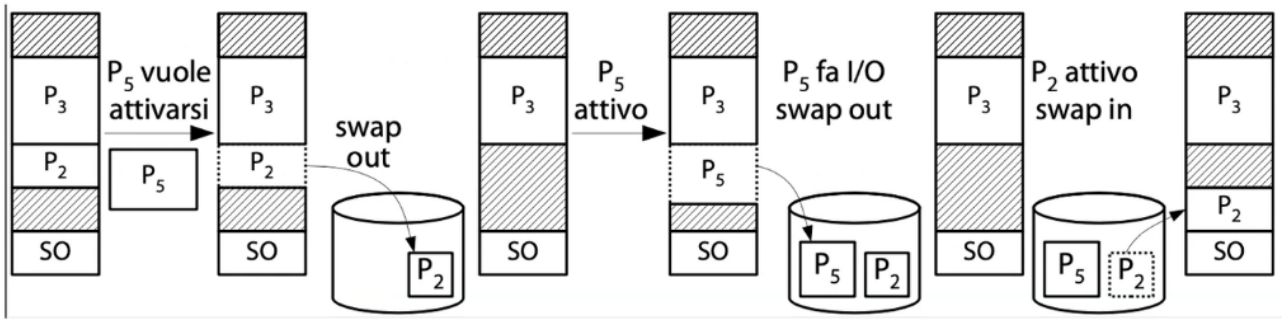
Rappresenta in modo astratto e logico la memoria cetrale di un calcolatore

Memory management unit

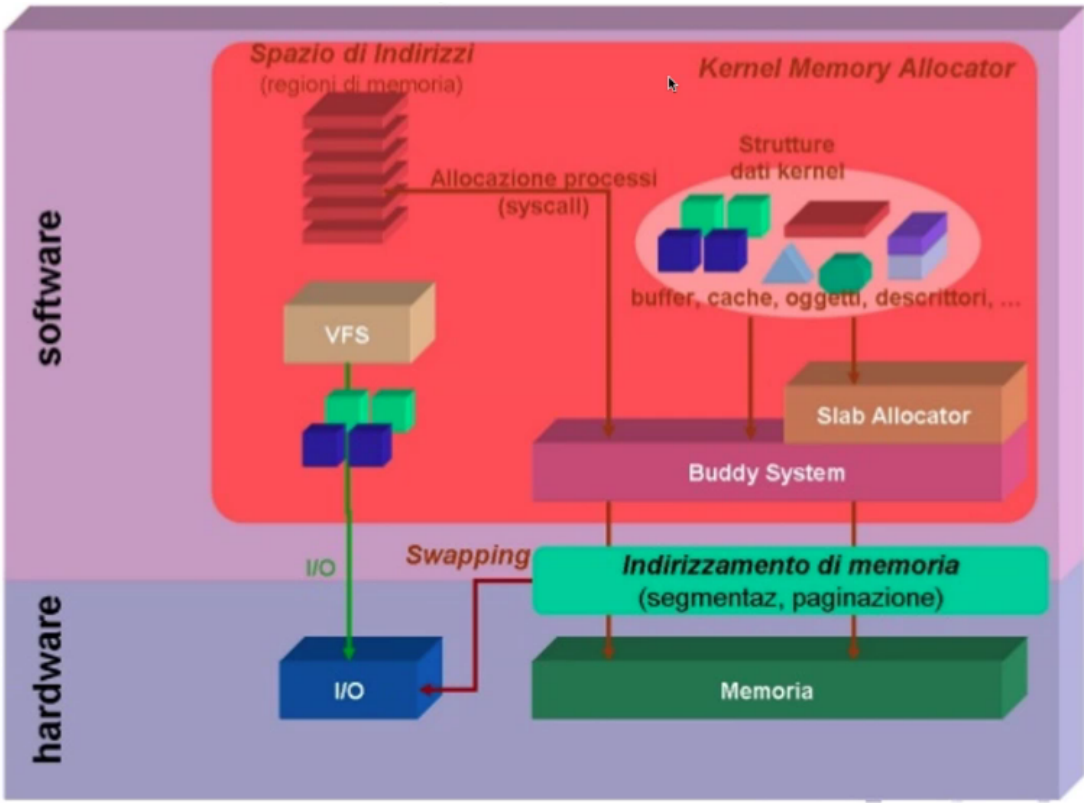
Gestisce le richieste di memoria delle applicazione traducendole in istruzioni da impartire all'unità HW di gestione della memoria

Vantaggi:

- Esecuzione concorrente dei processi;
- Esecuzione di applicazioni con richieste di memoria maggiori della memoria fisica;
- Condivisione da parte dei processi di una singola immagine di memoria di una libreria o del programma;
- Programmi rilocati



3.6 Memoria virtuale in Linux



Buddy System → tecnica di allocazione dinamica della memoria che divide la memoria in partizioni per soddisfare una richiesta di memoria nel miglior modo possibile.

Slab allocation → tecnica di allocazione della memoria per una migliore allocazione degli oggetti. Riduce la frammentazione causata dalla allocazione e della deallocazione

Kernel Memory Allocator

È la componente del kernel che cerca di soddisfare richesta di aree di memoria provenienti dal kernel stesso e dalle applicazioni

- Il KMA risponde a richieste di memoria provenienti da:
  - altri sottosistemi del kernel per uso del SO;
  - dai programmi utenti per mezzo di system call.
- Un KMA dovrebbe avere queste caratteristiche:
  - essere veloce;
  - minimizzare sprechi di memoria;
  - ridurre la frammentazione;
  - cooperare con gli altri sottosistemi di memoria per ottenere e rilasciare pagine

3.7 Spazio di indirizzi di un processo

Lo spazio di indirizzi virtuali che ogni processo può usare è diverso dall'insieme degli indirizzi fisici che usa ad ogni istante

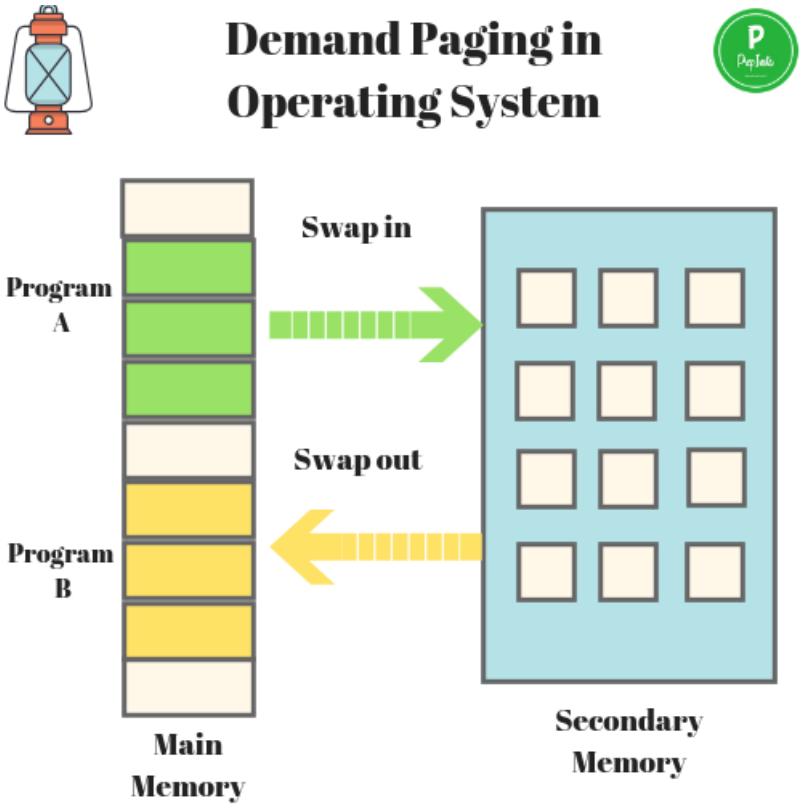
Il kernel rappresenta lo spazio di indirizzi come lista di descrittori di regioni di memoria

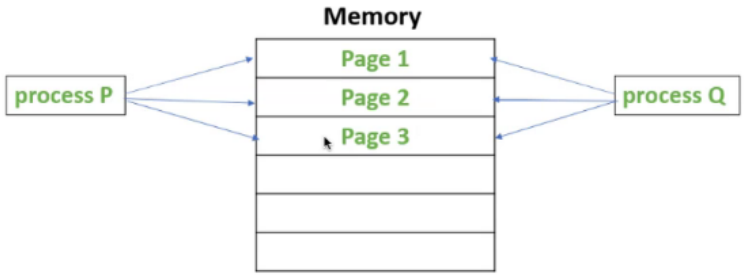
Alla creazione di un processo il kernel gli associa uno spazio di indirizzi che include regioni di memoria per:

- codice eseguibile;
- i dati inizializzati (e non) del programma;
- lo stack del programma;
- il codice eseguibile e i dati delle librerie condivise;
- lo heap (memoria dinamicamente richiesta dal processo);
- mapping in memoria di file usati dal programma

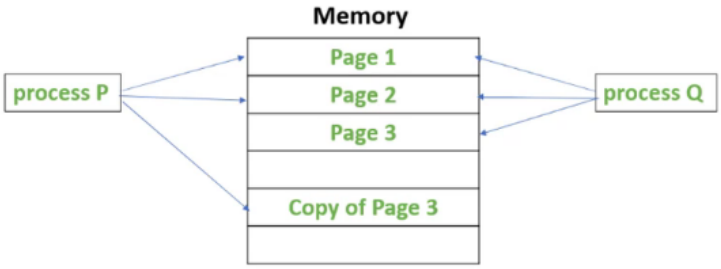
I moderni SO Unix usano strategie efficienti di allocazione di pagine basate sul Demand Paging e Copy-on-Write

Demand Pagin e Copy-on-Write a confronto





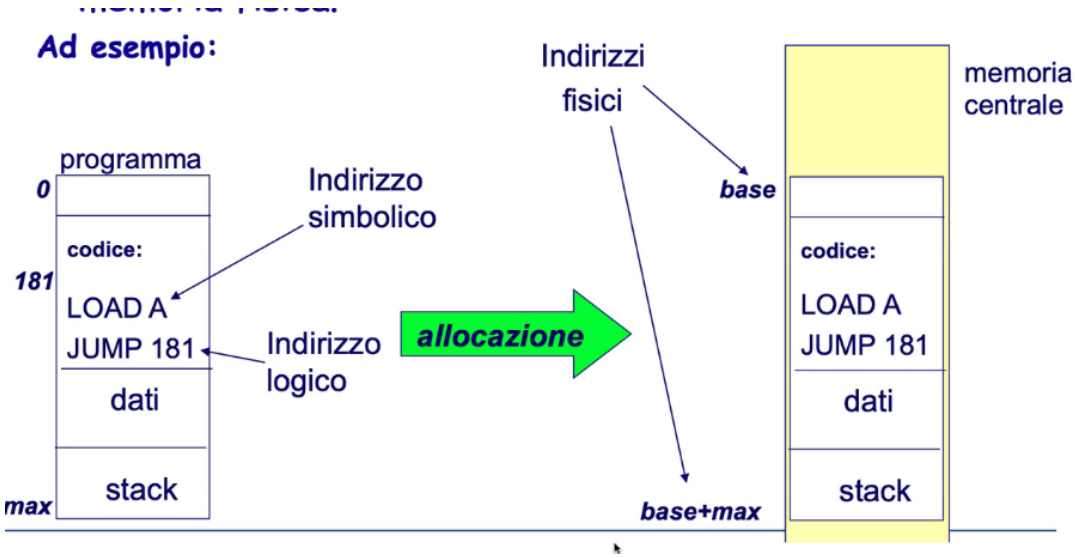
Before process P modifies Page 3



After process P modifies Page 3

Con la fork padre e figlio non lavorano sullo stesso spazio di indirizzamento, ma il figlio lavora su una copia dei dati del padre (a differenza dei thread che condividono la stessa memoria)

Ogni processo dispone di un proprio spazio di indirizzamento logico `[0,max]` che viene allocata nella memoria fisica



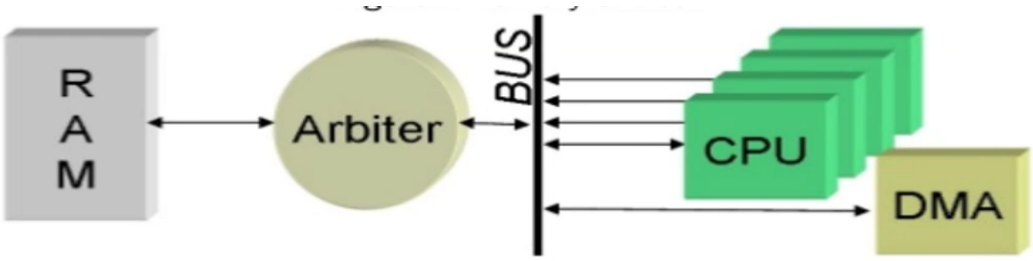
4 Lezione del 15-03

4.1 Swapping

Meccanismo HW e SW che consente di estendere la memoria centrale utilizzando spazi aggiuntivi su supporti fisici. Lo swapping è basato sulle pagine (swapped-out)

4.2 Memory Arbiter

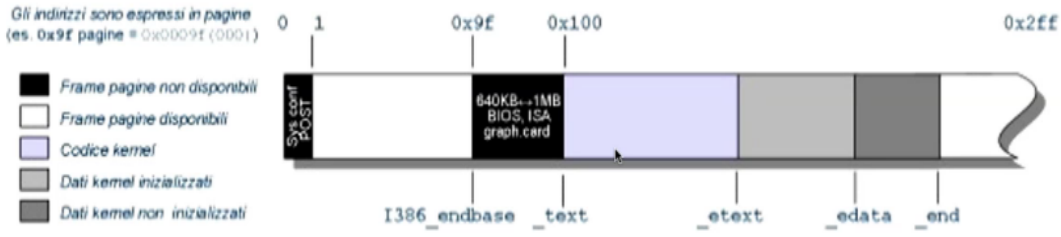
Un circuito HW che serializza (disciplina) gli accessi write-read alla ram da parte delle CPU (o dalla CPU e del DMA (accedere direttamente alla memoria interna per scambiare dati, in lettura e/o scrittura, senza coinvolgere CPU))



4.3 La RAM nei sistemi UNIX

È divisa in 2 sezioni:

- pochi mb per immagazzinare l'immagine del kernel;
- Il resto viene gestito dal sistema di memoria virtuale come memoria dinamica
  - Viene usata dal kernel per una serie di operazioni diverse



Tutto l'insieme dei meccanismi HW e SW per tradurre l'accesso ad un indirizzo di memoria virtuale (logico) nel corrispondente acccesso alla locazione fisica in RAM.

Il SO non deve occupari di tutti gli aspetti dell'accesso alla memoria fisica (nei moderni microprocessori esiste supporto HW per la traduzione e meccanismi hHW per l'individuazione e segnalazione di errore)

4.3.1 Risultati desiderati

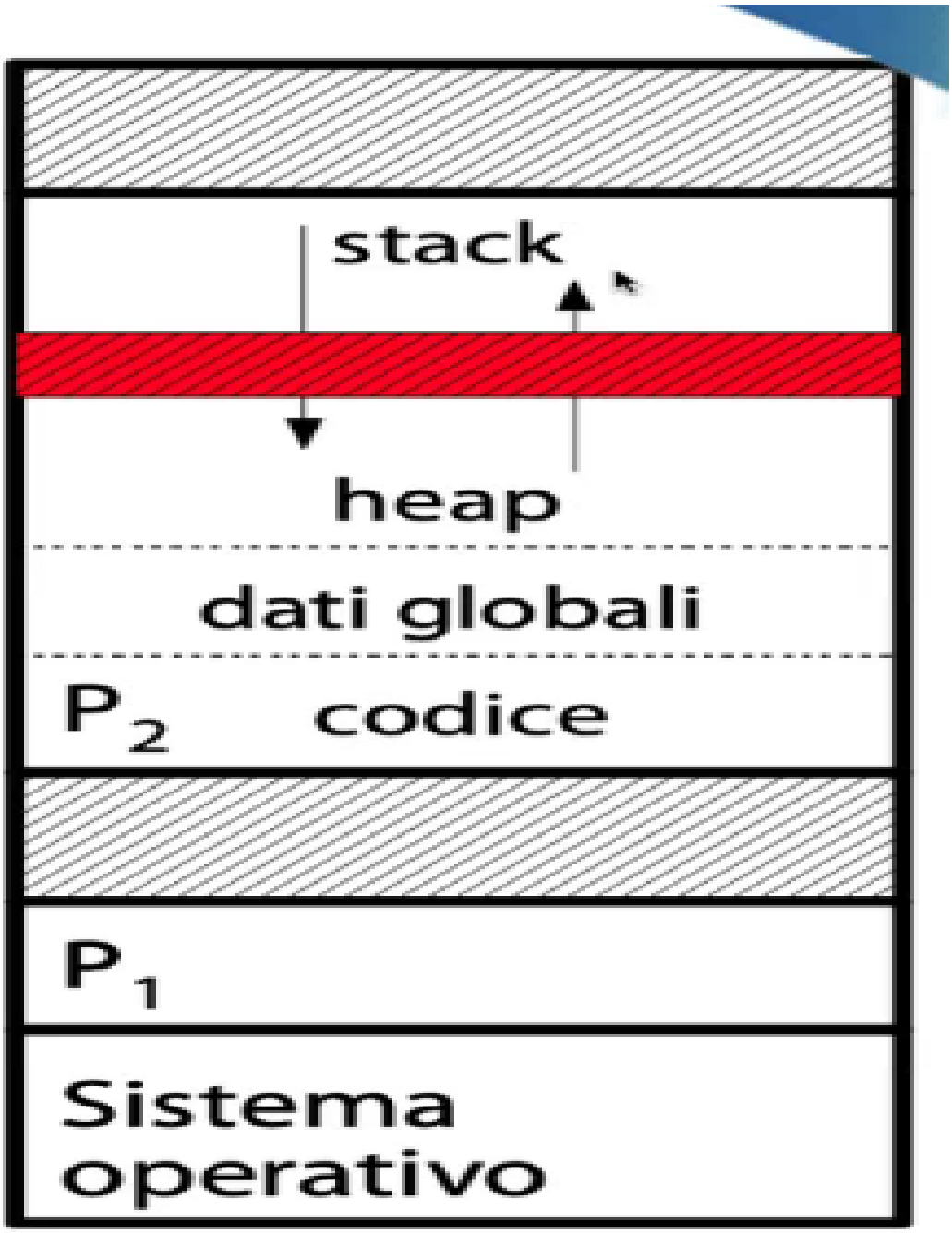
- La partizione di un processo deve essere sempre collocata in una porzione di memoria continua → swap di porzioni mirate della memoria di un processo
- Necessità di tenere lontani heap e stack
- Si deve impedire ad un programma l'accesso alla memoria di un altro programma
- Si deve impedire o limitare il buffer overflow

4.3.2 Stack & heap

Stack → heap: le strutture dati in cima all'heap contengono valori dello stack → corruzione dei dati sull'heap

Heap → stack: gli ultimi record di attivazione sono sovrascritti con valori provenienti dall'heap → il programma salta ad una locazione più o meno casuale (buffer overflow)

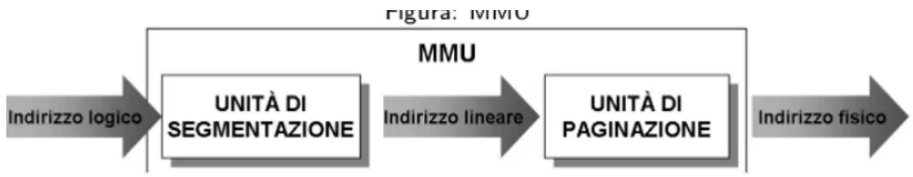
Se i dati sull'heap sono scelti bene, è possibile eseguire codice arbitrario

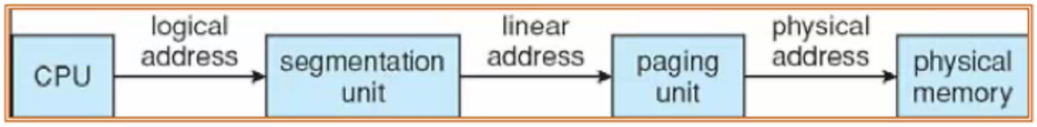

stack
heap
dati globali
P2 codice
P1
Sistema operativo

4.4 Memory Management Unit (MMU)

Componente della CPU che traduce indirizzi virtuali in fisici:

- Usa una circuitistica HW detta Unità di Segmentazione per tradurre indirizzi logici in indirizzi lineari
- Una una circuitistica HW detta Unità di Paginazione per tradurre indirizzi lineari in indirizzi fisici





4.5 Segmentazione

Schema di gestione della memoria che implementa una visione del programma per "blocchi funzionali"

4.6 Indirizzo logico

Un numero binario a 48 bit formato da un selettore di segmento (16 bit) e un offset (32 bit)

4.6.1 Segment selector

Identifica il segmento a cui appartiene l'indirizzo di memoria. Diviso in

- TI (1 bit) → indica in quale tabella è immagazziona il descrittore di segmento
  - GDT → tabella dei descrittori globali
  - LDT → tabella dei descrittori locali
- indice (13 bit) → posizione descrittore in tabella (GDT o LDT);
- RPL (2 bit) → flag che indica il CPL (current privilege level) che aveva la CPU quando il segmento viene caricato;
- i 6 registri di segmento (cs, ss, ds, es, fs, gs) contengono i selettori di segmento
  - i rimanenti bit non sono usati (cs che neusa 2 per il CPL)
    - cs → code segment;
    - ss → stack segment;

* ds → data segment;
* es → extra segment;
* fs, gs → inseriti per operazioni future;

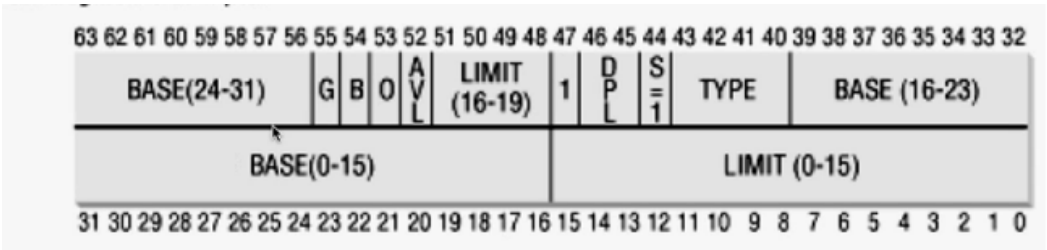
4.6.2 Offset

Indica la posizione dell'indirizzo rispetto all'inizio del segmento

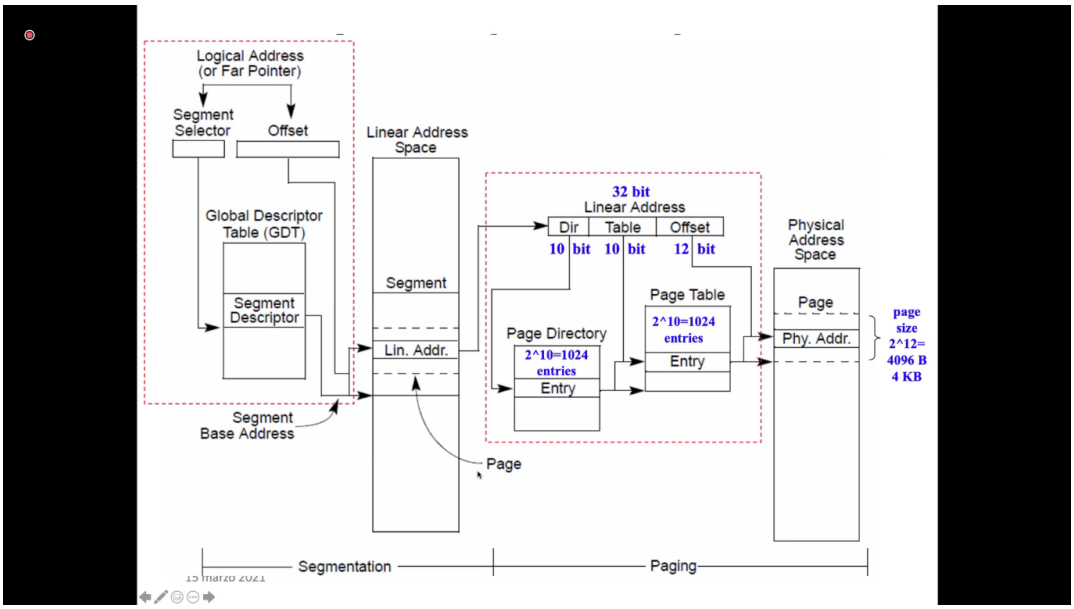
4.6.3 Descrittore di segmento

Dato di 64 bit che descrive le caratteristiche del segmento:

- BASE → indirizzo lineare base del primo byte del segmento;
- LIMIT → dimensione del segmento;
- TYPE → tipo di segmento;
- DPL (Descriptor Privile Level) → livello minimo di privilegi per accedere al segmento;
- bit 47° → 1 se presente in memoria, 0 altrimenti



4.6.4 Traduzione degli indirizzi logici in indirizzi fisici



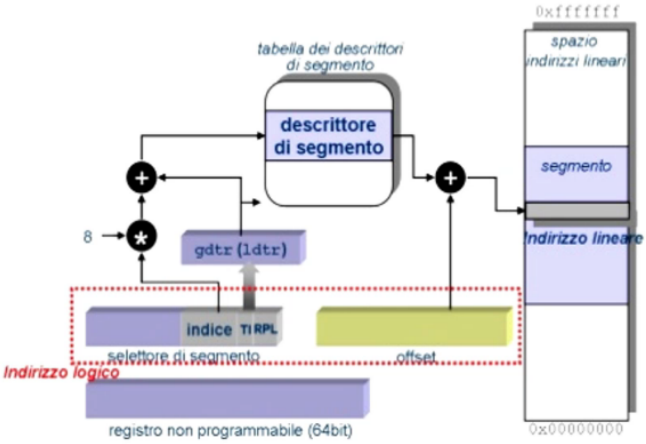
Ogni registro ha un registro non programmabile (con lo scopo di velocizzare la traduzione)

In esso è caricato il descrittore del segmento quando, per la prima volta, è caricato il selettore di segmento nel corrispondente registro segmento

Se il contenuto di un registro segmento cambia (flush), cioè un nuovo selettore è caricato, la segmentazione passa per la GTD (o LDT). Nel caso contrario (segmentazione diretta) non passa per la GTD (o LDT)

1. il valore `TI` determina se il descrittore si trova in GDT o LDT
2. l'indirizzo base della GDT (LDT) è nel registro `gdtr` (`ldtr`)
3. l'indice×8+ind.base = posiz. descr. in GDT (o LDT). Copia descr. in r.n.p.
4. dal descrittore si ricava l'indirizzo (lineare) base del segmento
5. ind. base segmento + offset = indirizzo lineare tradotto

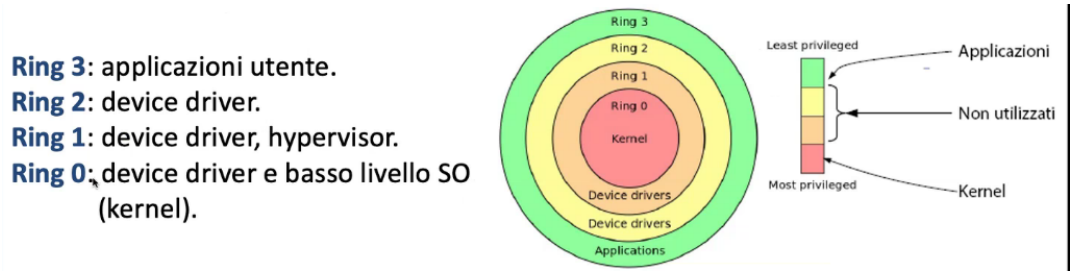
Figura: segmentazione: accesso in GDT



4.6.5 Privilegi di esecuzione

Meccanismo di difesa da istruzioni o accessi alla memoria causati da malfunzionamenti HW o comportamenti malevoli

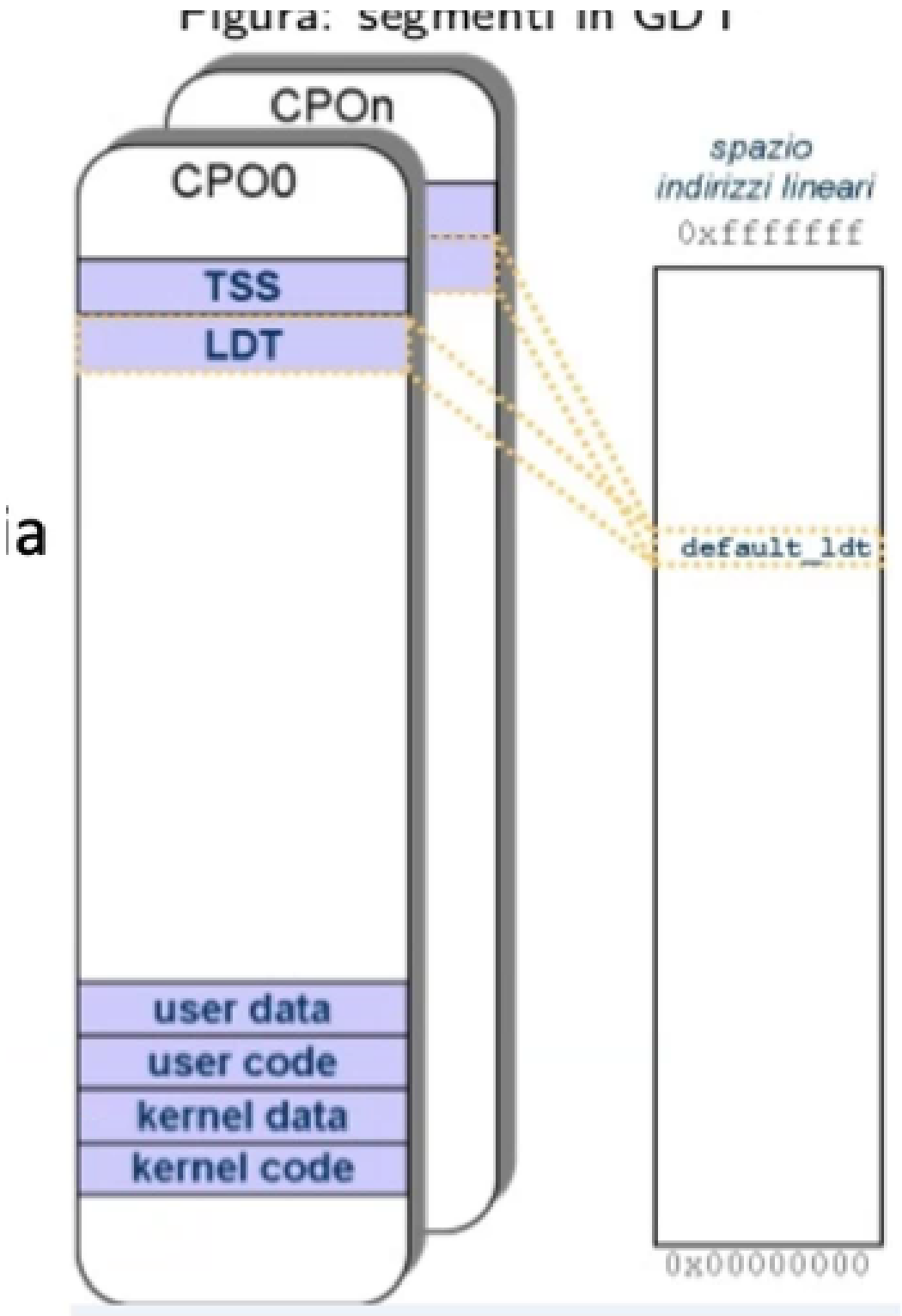
La CPU è dotata di 4 modi di utilizzo disposti ad anello



4.6.6 Tabelle dei Descrittori di Segmento

Vengono immagazzinati in GDT o LDT. Queste tabelle sono immagazzinate in memoria RAM.

I registri gdtr e ldtr contengono rispettivamente gli indirizzi fisici base della GDT e LDT



4.7 Segmentazione in sistemi LINUX

Linux usa principalmente la paginazione e in modo limitato la segmentazione (si limita il numero di flush)

4.7.1 I segmenti in Linux

1. Kernel code Segment Corrisponde ad un segmento di codice del kernel ed il suo segment selector è definito con la macro `__KERNEL_CS`
2. Kernel Data Segment Corrisponde ad un segmento di dati del kernel ed il suo segment selector è definito con la macro `__KERNEL_DS`
3. User Code Segment Corrisponde ad un segmento di codice condiviso da tutti i processi utente ed il suo segment selector è definito con la macro `__USER_CS`
4. User Data Segment Corrisponde ad un segmento di dati condiviso da tutti i processi utente ed il suo segment selector è definito con la macro `__USER_DS`

4.7.2 Spazio degli indirizzi lineari

Intervallo di indirizzi lineari da 0 (0x0000000) a 4G (0xFFFFFFF).

La dimensione massima dello spazio di indirizzi è determinata dall'architettura della CPU

4.7.3 Pagina di indirizzi lineari

Intervallo di indirizzi lineari di dimensione fissa (in Linux 4K); l'indirizzo base dell'intervallo è multiplo della dimensione.

4.7.4 Segmentazione vs Paginazione

Entrambi sono ridondanti → entrambi `partizionano/assegnano` la memoria fisica tra processi.

- La segmentazione assegma intervalli di indirizzi lineari a diversi ai processi;
- La paginazione mappa un intervallo di indirizzi lineari su intervalli di indirizzi fisici diversi

4.7.5 Page

Un blocco di dati che può essere scritto su un frame pagina.

Questo termine si riferisce sia alla pagina di indirizzo che alla pagina di dati memorizzati in RAM in corrispondenza.

Una frame pagina memorizza esattamente il blocco di dati (o istruzioni) contenuti in una pagina (di indirizzi lineari).

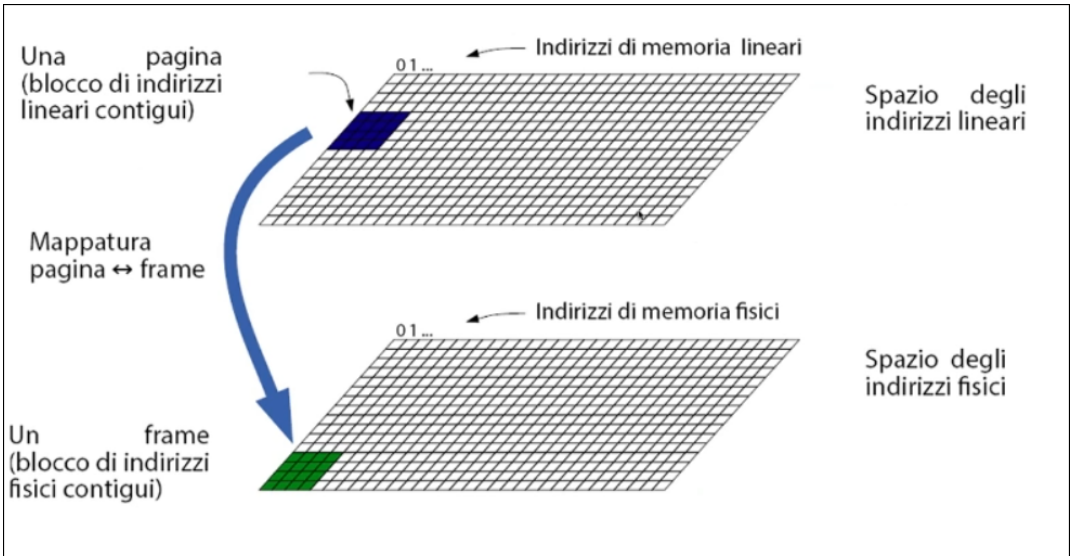
La paginazione mappa pagine di indirizzi lineari su frame pagine; una pagina è in RAM o in disco

1. Page Frame Area fisica (intervallo di locazioni contigue) di memoria RAM di dimensione fissata
2. Paging Unit Circuitistica della MMU che converte indirizzi lineari in fisici. Nella stessa pagina, indirizzi lineari contigui sono paginati su indirizzi fisici contigui.

   L'ultimo indirizzo di una pagina e quello della pagina successiva possono essere mappati su indirizzi fisici non contigui

   I permessi di accesso e i privilegi associati ad una pagina valgono per tutti gli indirizzi in essa contenuta.

   La paginazione verifica anche che la richiesta di accesso abbia privilegi e modalità compatibili con quelli associati alla pagina di appartenenza



3. La tabella delle pagine Ha una dimensione alta (viene mantenuta in RAM).

   Nei processori intel l'indirizzo fisico iniziale della tabella è salvato nel registro di controllo CR3.

   Come detto essendo gigantesca la sua dimensione si fa uso di uno **schema di paginazione gerarchico** → ciascun elemento della tabella delle pagine punta all'inizio di una tabella più piccola (che puntano ai frame fisici)

4.7.6 Tipi di Paginazione

- A 2 livelli (sistemi a 32 bit);
- A 3 livelli (in quanto il precedente non è più sufficiente);
- A 4 livelli (usati dai sistemi Linux), con 4 tipi di tabelle
  - Page Global Dir (PGD);
  - Page Upper Dir (PUD);
  - Page Middle Dir (PMD);
  - Page Table (PT).

4.7.7 TLB

Cache HW che velocizza la traduzione di indirizzi lineari in fisici

La prima volta che un indirizzo lineare è tradotto, l'indirizzo fisico corrispondente è registrato in un elemento della TLB. Per usi successivi dello stesso indirizzo linearre si userà l'indirizzo fisico presente in TLB.

Ogni CPU ha la propria TLB → sincronizzazione non necessaria:

- Processi che seguono su diverse CPU possono usare stessi indirizzi logici corrispondenti a indirizzi fisici diversi;
- TLB funziona da cache per gli elementi della Page Table;
- Se cambia l'indirizzo fisico di una frame pagina (swapping), l'elemento della TLB corrispondente è invalidato;
- Se cambia il valore del registro CR3 (PD) tutta la TLB viene invalidata

5 Lezione del 19-03

5.1 La programmazione di sistema

Consiste nell'utilizzare l'interfaccia di system call fra il kernel e le applicazioni che girano sotto UNIX.

5.1.1 System call

Per i programmi C, le system call sono come funzioni. Ogni system call ha un prototipo:

- `pid_t` (0 se padre, altro valore se figlio)

Tra le system call ricordiamo:

- getpid;
- getppid;
- getgrp;

Che forniscono degli attributi dei processi (PID, PPID, gruppo etc)

- fork → crea un processo figlio duplicando il chiamante;
- exec → trasforma un processo sostituendo un nuovo programma nello spazio di memoria del chiamante;
- wait → permette la sincronizzazione dei processi;
- exit;

5.1.2 Fork

Il prototipo è `pid_t` *fork()*, dove:

- `pid_t` è definito negli include di sistema.

Il valore restituito da fork può essere usato per distinguere tra processi genitore e processo, infatti al genitore viene restituito il PID del figlio, al figlio viene invece restituito 0. Restituisce 0 al figlio e pid al padre, oppure -1 (solo al padre) in caso di fallimento

5.1.3 Processo Zombie

Un processo diviene zombie dopo essere terminato e prima che il processo padre abbia eseguito una *wait* su quel figlio

5.1.4 Esempio

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(){
    pid_t pid;
    int status;
    pid = fork();
    if (pid)
        sleep(2);
        pid = wait (&status);
        if (WIFEXITED (status)) {
            printf ("stato %d\n", WEXITSTATUS(status));
        } else{
            printf ("Processo %d, figlio. \n ", getpid());
            _exit(17);
        }
}
```

5.2 Gestione dei processi in Linux

Si definisce processo una qualsiasi istanza di programma in esecuzione.

Supporto di multithreading, attraverso i lightweight process (queste condividono le risorse) → in questo modo vengono supportati i thread.

5.3 Process Descriptor

Il kernel monitora costantemente l'attività dei processi mediante una struttura di tipo `task_struct` che prende il nome di Process Descriptor

Questo descrittore contiene una serie di puntatori a strutture secondarie (che contengono le relative informazioni)

Il kernel contiene la lista dei processi in una lista doppiamente linkata chiamata task list

5.4 Stati da un processo

- `TASK_RUNNING`;
- `TASK_INTERRUPTIBLE`;
- `TASK_UNINTERRUPTIBLE`;
- `TASK_STOPPED`;
- `TASK_TRACED`;
- `EXIT_ZOMBIE`;
- `EXIT_DEAD`.

5.5 Identità di un processo

Qualunque processo in Linux è caratterizzato da un process descriptor ed il puntatore a tale struttura prende il nome di process descriptor pointer. Linux identifica i processi assegnando loro un Process ID (PID) che viene memorizzato nel campo `pid_field` del process descriptor.

I PID vengono assegnati in ordine crescente e riutilizzati solo dopo aver raggiunto il valore massimo. Linux tiene traccia dei PID nuovamente disponibili in un `pidmap_array`

5.6 Relazioni fra processi

I principali sono:

- Relazione `padre/figlio;`
- Relazione processi fratelli.

Ma ne esistono molti altri in base al condizione che andiamo a verificare.

5.7 Le liste di processi RUNNING

Nelle precedenti versioni di linux, la selezione del processo da eseguire avveniva scorrendo una *runqueue* alla ricerca del processo con priorità più alta.

Le attuali versioni implementano un algoritmo di selezione in tempo costante, mantenendo k liste di processi, dove k rappresenta la priorità (un valore intero tra 0 e 139)

5.8 I file hash

Si servono di una funzione di hash o funzione di randomizzazione, tale che applicata al valore di uno dei campi restituisce l'indirizzo del blocco in cui è memorizzato il record. Consentono operazioni di ricerca molto efficienti.

Per file interni, l'hash è implementato con una tabella di hash.

5.8.1 Funzione di accesso h

Definita come:

$$h{:}K \rightarrow \{0,1,2,\ldots,\ m\text{-}1\}$$

Dove:

- K → insieme dei valori distinti che il campo chiave può assumenre;
- m → dimensione del vettore in cui si intende memorizzare gli elementi della tabellla

5.8.2 Divisione

Definita come:

$$h(k) = k \bmod |T|$$

Presenta una bassa complessità, ma può generare fenomeni di agglomerazione

5.8.3 Risoluzione delle Collisioni

Esistono diverse strategie per la risoluzione delle collisioni:

- Concatenamento → alla i-esima posizione della tabella è associata la lista degli elementi tali che *h(k)=1*;
- Indirizzamento aperto → tutti gli elementi sono contenuti nella tabella.

La scelta della strategia influisce sull'efficienza delle operazioni di inserimento, ricerca e cancellazione.

5.8.4 Tabella Hash per la runqueue

Il Kernel mantiene quattro tabelle hash, che gli permettono di collegare il PID di un processo al suo process descriptor

Le collisioni sono gestite mediante liste concatenate

5.9 Organizzazione dei processi

La runqueue contiene solo i processi nello stato `TASK_RUNNING`.

I processi nello stato `TASK_STOPPED` `EXIT_DEAD` `EXIT_ZOMBIE` non vengon raggruppate in nessuna lista.

I processi che si trovano nello stato di `TASK_INTERRUPTIBLE` e `TASK_UNINTERRUPTIBLE` vengon raggruppati a seconda dell'evento per cui sono stati accodati

Poichè lo stato di un processo non è sufficiente a decidere la politica di risveglio da adottare si fa uso delle wait queue

5.9.1 Wait queue

Usate dal kernel per gestire diverse situazioni:

- Gestione degli interrupt;
- Sincronizzazione dei processi;
- Timing.

Sono implementate mediante liste doppiamente concatenate, i cui elementi puntano a process descriptors. Sono fornite inoltre di uno spinlock poichè possono essere utilizzate sia delle principali funzioni del kernel, che dai gestori degli interrupt

5.9.2 Struttura di una Wait queue

Il campo `task_list` contiene il puntatore alla lista di processi che attendono nella coda. La decisione di risvegliare un processo in attesa in una coda dipende da diversi fattori. In particolare bisogna valutare se esso attende per una risorsa che può essere acceduta in modo esclusivo.

Esistono due tipi di processi sleeping:

- Esclusivi $\rightarrow$ identificati dal campo flags impostato a 1;
- Non esclusivi $\rightarrow$ identificati dal campo flags impostato a 0;

5.10 Duplicazione delle risorse

I SO Unix si basano sulla generazione a catene di processi per servire le richieste dell'utente. Unix tratta i processi allo stesso modo.

Le risorse di un processo sono duplicate e assegnate ai figli. Questo modalità è inefficiente.

Nei moderni sistemi Linux questo problema viene risolto attraverso diverse tecniche:

- Copy-or-Write;
- Lightweight processes (threads);

5.10.1 Chiamate di sistema in Linux

Ne esistono 3:

- fork() → valore di ritorno
  - nel contesto di esecuzione del padre il PID del processo figlio;
  - nel contesto di esecuzione del figlio ritorna 0;
  - padre e figlio non condividono lo spazio di indirizzi
- vfork() → analogo al fork;
- clone) → valore di ritorno
  - PID del processo creato;
  - -1 in caso di fallimento.
  - padre e figlio condividono le risorse indicate

Tutte e 3 invocano la stessa funzione di servizio `do_fork()`

5.11 Bomba fork

Attacco di tipo denialof service contro un computer che utilizza la funzione fork.

Es in bash

```
:() { : | : & };:
```

5.12 Terminazione di un processo

Un task termina su sua richiesta (sincrono) quando:

- finisce di eseguire il proprio codice
  - chiamata esplicita o implicita della funzione di sistema *exit()* (o `pthread_exit()`)

Un task non termina su sua richiesta (asincrono) quando:

- riceve segnali di terminazione;
- avviene un'eccezione.

In tutti i casi è il Kernel che ne forza la terminazione:

- rilasciando tutte le risorse del task;
- rimuovendo il task dalle liste del sistema e deallocandole il descrittore + KMS

5.13 I thread

Rappresenta uno dei possibili sottoprocessi eseguibili all'interno di esso. Sono quindi parti di un processo che vengono eseguite in maniera concorrente ed asincrona

Nei thread vengono condivise le medesime informazioni di stato, la memoria ed altre risorse di sistema

La differenza è il meccanismo di attivazione:

- l'operazione di attivazione di un processo è un operazione onerosa;
- l'operazione di attivazione di un thread richiede meno risorse.

5.13.1 Kernel-level e user-level thread

Thread a livello utente

Sono gestiti come uno strato separato sopra il nucleo del SO, e realizzati tramite una libreria di funzioni per la creazione, lo scheduling, e la gestione, senza alcun intervento diretto del nucleo

Thread a livello del nucleo

Gestiti direttamente dal SO

Distinguiamo 3 tipi di modelli:

1. Modello da molti a uno Fa corrispondere molti thread a livello utente a un singolo thread al livello del nucleo
2. Modello da uno a uno Fa corrispondere ciascun thread a livello utente con un thread al livello del nucleo.
3. Modello da molti a molti Fa corrispondere più thread a livello utente con un numero minore o uguale di thread al livello del nucleo

5.13.2 Clone()

Esiste la possibilità attraverso la system call clone() di generare thread. Questa accetta come parametro un insieme di flag che definiscono quante e quali risorse del processo genitore debbano essere condivise dal processo figlio, come:

- `CLONE_FS` → le informazioni sul file system sono condivise;
- `CLONE_VM` → condivisione dello stesso spazio di memoria;
- `CLONE_SIGHAND` → condivisione dei gestori dei segnali;
- `CLONE_FILES` → condivisione dell'insieme dei file aperti

5.13.3 Pthreads

Lo standard POSIX (Portable Operating System Interface for UniX) → definisce l'API per la creazione e la sincronizzazione dei thread.

Non si tratta di una realizzazione ma di una definizione del comportamento dei thread.

I progettisti di SO possono realizare le API così definite come meglio credono

6 Lezione del 22-03

6.1 Kernel thread

Linux delega alcune attività critichee e periodiche a task, detti **kernel threads**:

- Flush di cache disco;
- Swapping out di pagine non utilizzate;
- Servizi di connessione rete;
- ...

I kernel threads:

- Eseguono solo in kernel mode;
- Sono schedulati in modo intermittente;
- Eseguono una funzione del kernel specifica.

La funzione `kernel_thread()` ha il compito di creare un nuovo kernel thread. Riceve come parametri la funzione del kernel da eseguire e i suoi comandi.

6.1.1 Processi

- Processo 0 (swapper) → alloca diverse strutture dati referenziate dai processi;
- Processo 1 (initi) → creato da swapper, è l'antenato di tutti i processi;
- kswapd → recupera memoria;
- ksoftirqd → esegue tasklet di interruzioni;
- pdflush → esegue flush di buffer dirty su disco.

6.1.2 Process switch

Si intende l'attività del kernel di sospendere l'esecuzione di un processo per sostituirla con quella di un processo differente.

Viene effettuato sempre in Kernel mode.

Gli elementi coinvolti sono 2:

- Contesto HW;
- politiche di avvicendamento dei processi.

Contesto SW

Occorre ripristinare:

- Lo stato dei registri della CPU;
- risorse utilizzate;
- stato della memoria

Alcune informazioni sono salvate in memoria (`task_struct`), altre nella CPU (`registri`).

Contesto HW

Conservato in:

- process descriptor del processo;
- Kernel Model stack del processo.

6.2 Scheduling dei processi

Linux è un sistema multi-tasking e time-sharing.

Gli aspetti importanti legati allo scheduling sono 3:

- politiche di scheduling;
- algorimti di scheduling;
- strutture dati e le funzioni che implementano lo scheduling;

6.2.1 Scheduling policy

- Tempo di risposta breve;
- elevato throughput per i processi in background;
- evitare la starvation di processi;
- convisione di risorse fra processi ad alta e bassi priorità.

In linux il tempo di esecuzione è suddiviso in *slice* → al termine di questo è soggetto ad un process switch.

Diversi modi di categorizzazione dei processi:

- `I/O` bound;
- CPU bound.

Possono essere classificati in:

- interattivi → richiedono interazioni con l'utente;
- batch → non richiedono interazioni con l'utente e sono spesso eseguiti in modalità background;
- Real-time → vincoli di scheduling molto stretti e non possono essere bloccati da processi con priorità inferiore.

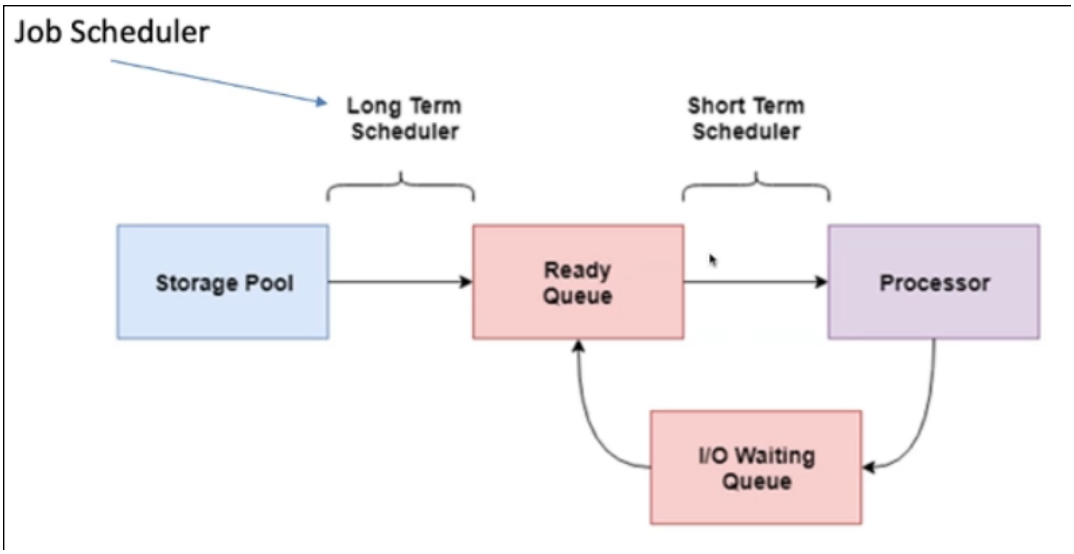
6.2.2 Single e multi-core

Il passaggio da single a multi-core è avvenuto quando ci si è resi conto che non conviene aumentare ulteriormente la frequenza di un core unico a causa dell'eccessivo consumo (fino a 100W) e calore dissipato. Il multi-core aggira questo problema, ma richiede politchè ed algoritmo di scheduling in grado di sfruttare al meglio tale possibilità

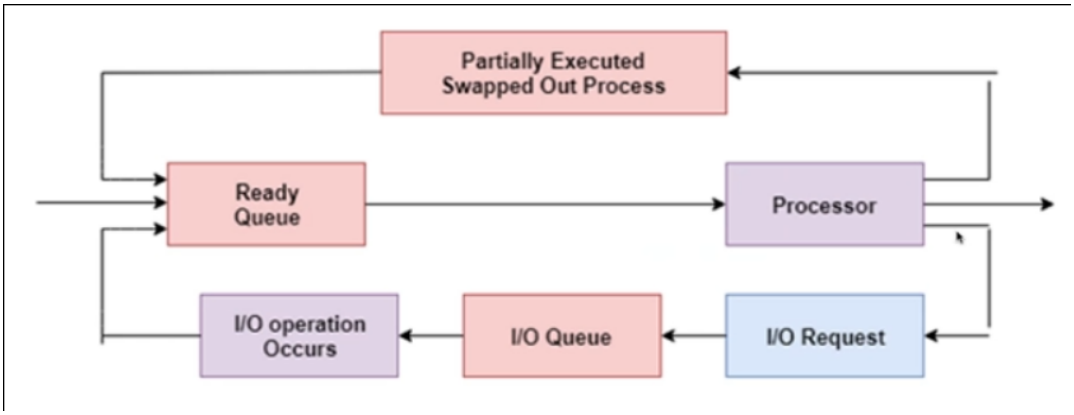
6.2.3 Tipologie di Scheduling

- Long-Term scheduling → un nuovo processo viene aggiunto all'insieme dei processi che devono essere eseguiti;
- Middle-Term scheduling → un nuovo processo viene aggiunto all'insieme dei processi che sono parzialmente o completamente in memoria;
- Short-Term scheduling → seleziona direttamente un processo disponibile da sottomettere ad un processore.

Long-Term e Short-Term scheduling



Middle-Term scheduling



6.2.4 Caratteristiche di uno scheduler

- Trasparente al programmatore;

- Minimizzare i tempi di context switch;
- Bilanciare il carico in maniera equa;
- Assicurare che se vi sono task con priorità maggiore, questi vengano eseguiti prima degli altri;
- I processi interattivi devono rispondere velocemente alle richieste da parte dell'utente;
- Ai job che lavorano in background deve comunque essere garantita la possibilità di portare a termine il loro lavoro in tempo ragionevoli;
- Nessun processo dovrebbe soffrire di starvation.

6.2.5 Prelazione

Un processo può essere prelazionato se:

- nella run-queue entra un processo con priorità maggiore di quello corrente;
- esso ha esaurito il suo quanto di tempo

Un processo prelazionato è diverso da un processo sospeso → resta in uno stato di `TASK_RUNNING`.

Critico per sistemi a prelazione è la durata del quanto di tempo:

- Se il quanto è troppo breve, i process switch sono molto frequenti causando un elevato overhead;
- Se il quanto è troppo lungo, non si ha più la percezione da parte dell'utente che i processi siano eseguiti in modo concorrente.

La scelta della durata del quanto è un compromesso. Linux sceglie la più lunga durata che non degradi le prestazioni del sistema.

6.2.6 Algoritmo di scheduling

Basata sulle code di priorità e garantisce prestazioni *O(1)* sia al crescere del numero di processi, che del numero di CPU (ciascuna presenta la propria coda).

Lo scheduler ha sempre almeno un processo in stato `TASK_RUNNING` da selezionare: lo *swapper*.

1. `SCHED_FIFO`

   Quando lo scheduler assegna ad una CPU, lascia il process descriptor nella sua posizione corrente all'intero della lista della run-queue (Realtime FIFO scheduler).

   Il sistema non interromperà un processo in esecuzione con FIFO ad eccezione dei seguenti casi:

   - Un altro processo FIFO con priorità maggiore diventa pronto;

- Il processo è bloccato da una `I/O`;
- Il processo rilascia volontariamente il processore.

Quando un processo viene interrotto, questo viene inserito nella coda associata alla sua priorità. Nel caso in cui più di un processo ha priorità alta, viene scelto il processo che ha atteso di più

2. `SCHED_RR`

   Quando lo scheduler assegna il processo ad una CPU, pone il process descriptor alla fine della lista della run-queue. (Realtime RR scheduler). A differenza di quelle FIFO nello `SCHED_RR` viene aggiunta di un quanto di tempo associato ad ogni thread

3. `SCHED_OTHER`

   Un processo time-shared convenzionale (Round Robin scheduling policy)

6.2.7 Scheduling di processi convenzionali

In Linux ogni processo ha una propria priorità statica. Il Kernel riserva 40 valori per rappresentare questa priorità (da 100 a 139) → all'aumentare del valore diminuisce il livello di priorità (I livelli da 0 a 99 sono riservate ai processi Real Time).

Un nuovo processo eredita sempre la priorità statica del padre, tuttavia un utente può cambiare il nice del processo attraverso le chiamate a sistema *nice()* e *setpriority()*

6.2.8 Quanto temporale di base

Dalla priorità statica lo scheduler calcola il **base time quantum**, ossia la durata del quanto temporale di esecuzione di un processo. Terminato quest ultimo il processo viene fermato a favore dei processi concorrenti che possono eseguire.

A ciascun processo vengono assegnati quanti temporali di lunghezza variabile a seconda delle caratteristiche del processo. Il calcolo del nuovo quanto viene effettuato con la seguente equazione:

$$\text{quanto temporale}_{\text{ms}} = (140 \text{ - priorità statica}) \text{ x } 20 \text{ se la priorità statica} < 120$$

$$\text{quanto temporale}_{\text{ms}} = (140 \text{ - priorità statica}) \text{ x } 5 \text{ se la priorità statica} \geq 120$$

6.2.9 Durata del quanto temporale di base

Ogni processo è caratterizzato anche da una priorità dinamica che va da 100 (priorità massima) a 139 (priorità minima). Questa priorità è quell'attributo considerato dallo scheduler nel momento in cui deve trovare il processo più adatto da eseguire.

$$\text{priorità dinamica} = \max(100, \min(\text{priorità statica - bonus} + 5{,}139))$$

Il bonus è una variabile compresa tra 0 e 10. Il valore che lo scheduler attribuisce a questa variabile → dipende dal valore assunto dal tempo medio di sleep (average sleep time)

Il rovescio della medaglia è la conversione di processi ad altra priorità, che nel caso di alti sleep time rischiano di diventare processi interattivi

6.2.10 Average sleep time

Tempo medio che il processo passa nello stato sleep. Non viene calcolato effettuando banalmente la media degli intervalli di tempo:

- Il tempo speso in sleep quando il processo è in `TASK_INTERRUPTIBLE` o `TASK_UNINTERRUPTIBLE` pesa diversamente;
- Diminuisce quando un processo è in esecuzione;
- Non può superare il valore di 1 secondo.

Un processo viene considerato interattivo se:

$$\text{priorità dinamica} \leq 3 \text{ x } \frac{priorità statica}{4+28}$$

o in maniera equivalente

$$\text{bonus -5} \geq \frac{priorità statica}{4+28}$$

7 Lezione del 26-03

7.1 Processi batch e interattivi

La natura di un processo rimane piuttosto invariata. Se effettivamente lo scheduler scegliesse sempre il processo miglore, si penalizzerebbe l'esecuzione dei processi a bassa priorità.

Per risolvere questa problematica il kernel attua due policy:

- processi attivi (*active processes*)
  - processi che non hanno ancora completato il loro quanto temporale
- processi scaduti
  - processi che hanno appena terminato il loro quanto temporale e finchè non si esauriscono i processi attivi, questi continueranno a non eseguire

Un processo batch se ha finito il suo quanto temporale viene sempre aggiunto alla lista di processi scaduti. Questo non è vero per un processo interattivo, ricalco del suo quanto temporale → resta nella lista dei processi attivi a meno che:

- il primo dei processi della lista expired ha aspettato oltre una determinata soglia;
- un processo scaduto ha priorità statica maggiore del processo interattivo
  - in questo caso lo scheduler è programmato per aggiungere alla lista dei processi scaduti

7.2 Scheduling dei processi Real-time

Priorità da 0 a 99. Questo tipo di processi sono sempre attivi. Lo scheduler scegli quello che risulta primo della lista di priorità a cui appartiene (di una data CPU). Questo tipo di processo viene rimpiazzato se:

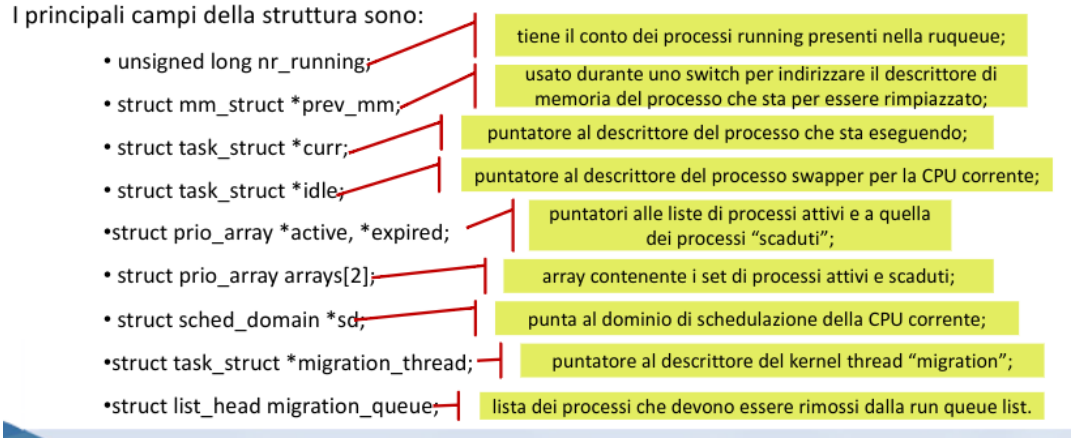
- subisce prelazione da un altro processo con maggiore priorità real-time;
- Esegue un operazione bloccante ed è messo in sleep;
- il processo è stato fermato o ucciso;
- Il processo rilascia volontariamente la CPU lanciando la chiamata di sistema `sched_yield()`;
- Il processo è in Round Robin real-time, e ultimato il suo quanto di tempo viene prelazionato.

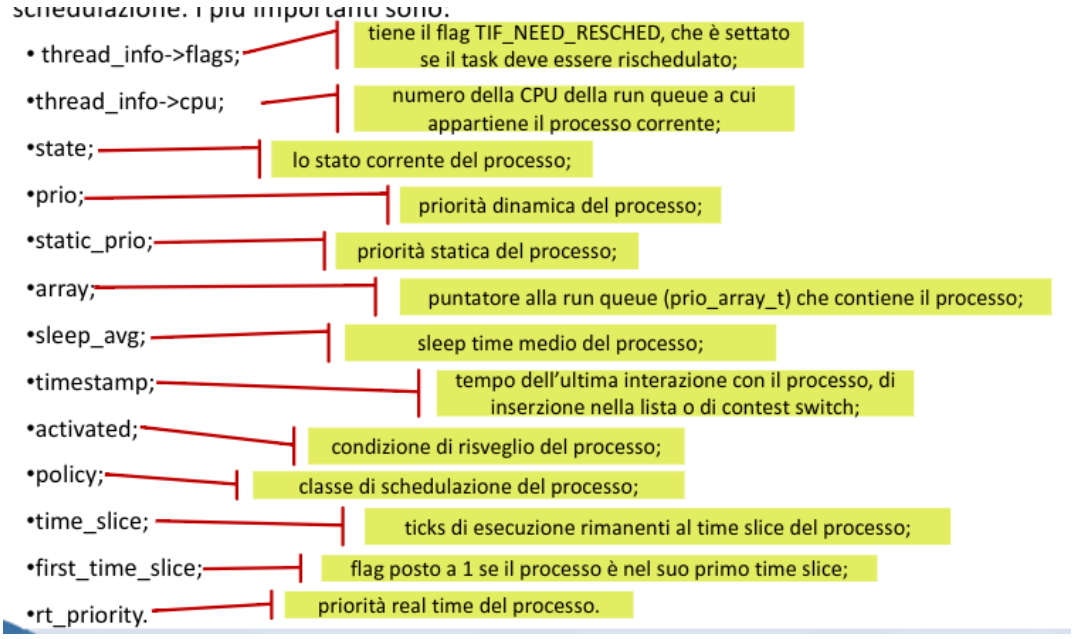
7.3 Strutture dati usati dallo scheduler

La struttura a run queue è la più importante fra quelle utilizzate dallo scheduler su Linux. Ogni CPU presente nel sistema ha la propria run queue.

La macro `this_rq()` tiene l'indirizzo della run queue della CPU locale, mentre la macro `cpu_rq(n)` fornisce l'indirizzo della run queue della CPU con indirizzo n.

7.3.1 Principali campi della struttura





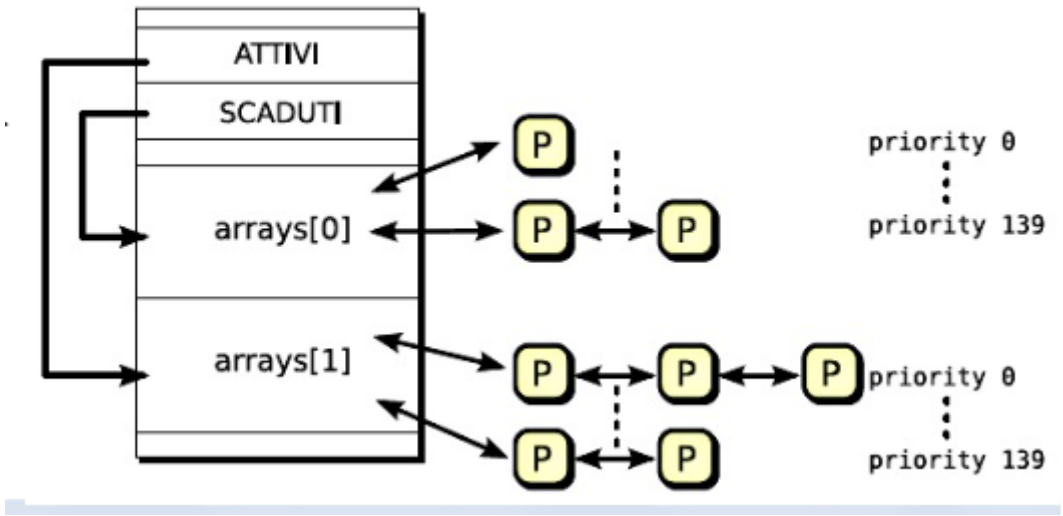
7.4 Scheduling dei processi nel dettaglio

Il campo più importante della struttura runqueue è quello legato alla lista di processi eseguibili (run queue).

Ogni processo runnable appartiene ad una e una sola run queue → finchè il processo rimarrà in quella lista, solo la CPU corrispondente a quella run queue potrà eseguirlo.

La run queue prevede inoltre due puntatori a delle strutture di tipo `prio_array` che detengono la lista dei processi attivi e scaduti.

Esse includono 140 nodi iniziali (heads) di doubly linked list (una per priorità), una mappa di priorità (priority bitmap) ed un contatore che conta i processi della struttura. Periodicamente i due puntatori vengono scambiati tra di loro.



7.5 Funzioni utilizzate dallo scheduler

Lo scheduler utilizza 5 funzioni fondamentali nella gestione dei processi:

- `scheduler_tick()` → utilizata per aggiornare il `time_slice` del processo current;
- `try_to_wake_up()` → risveglia un processo dormiente;
- `recalc_task_prio()` → aggiorna la priorità del processo;
- `schedule()` → seleziona il processo che deve eseguire;
- `load_balance()` → tiene le run queue dei vari processi bilanciate

7.6 Aggiornamento del time slice di processi Real-time

Per i processi real-time le istruzioni di aggiornamento si distinguono a seconda che il processo current sia:

- FIFO
  - lo scheduler non fa nulla (il processo non può subire preemption da un processo con priorità uguale o minore)
  - Non avrebbe senso tenere il suo contatore aggiornato
- RR
  - Se il quanto è scaduto dopo averlo decrementato
    - viene ricalcolato il time slice del processo;
    - viene azzerata la condizione di esecuzione nel primo time slice;
    - viene posoto il flag `TIF_NEED_RESCHED` ad 1.

Prima di uscire, la funzione pone current in coda ai processi dello stesso anello, rilascia il lock e forza l'invocazione dello scheduler quando l'esecuzione rientra dall'interrupt del timer

7.7 Rendere un processo running

I processi possono attendere la realizzazione di un certo evento, e dunque si sospendono in uno stato `TASK_INTERRUPTIBLE` o in uno stato di `TASK_UNINTERRUPTIBLE`.

Qualora un processo in uno di questi stati volesse risvegliarsi, la routine del kernel adibita a questa funzione dovrà utilizzare il metodo `try_to_wake_up`.

Il metodo riceve i seguenti parametri:

- il descrittore del processo da risvegliare;
- una maschera dello stato del processo che può essere risvegliato;
- un flag che proibisce al processo risvegliato di prelazionare il processo correntemente running

7.7.1 Funzione schedule()

Ha il compito di scegliere il processo più adatto all'esecuzione. Due tipo di invocazione:

- *modalità diretta* → si presenta quando un task deve sospendersi per mancanza di una risorsa;
- *modalità lazy* → attivando il flag `TIF_NEED_RESCHED` si forza l'invocazione dello scheduler al successivo ritorno in User Mode.

Modalità diretta

In questo caso la routine del kernel (a seguito di una chiamata di sistema) deve:

- Inserire current nella coda di attesa della risorsa associata che gli spetta;
- Cambiare lo stato di current in `TASK_INTERRUTIBLE` o

`TASK_UNINTERRUPTIBLE`, a seconda del processo;

- Invocare la funzione `schedule()`;
- Controllare periodicamente se la risorsa è disponibile;
- Quando la risorsa è disponibile, risvegliare il processo rimuovendolo dalla coda di attesa.

È il caso di alcuni driver che eseguono lunghe iterazioni. Questi task controllano ad ogni iterazione se il valore del flag di current è impostato su `TIF_NEED_RESCHED` , nel qual caso chiamano la funzione schedule() per rilasciare volontariamente l'uso della CPU.

Modalità lazy

Ci possono essere vari casi in cui si effettua una chiamata della funzione in questo modo:

- current ha ultimato il suo quanto temporale, `scheduler_tick()`

quindi setta il flag `TIF_NEED_RESCHED`;

- Un task, che è stato risvegliato dalla routine del kernel, ha

priorità maggiore del processo current, in questo caso l'azione è compiuta dalla funzione `try_to_wake_up()`;

- Quando è invocata la chiamata di sistema

`sched_setscheduler()`.

7.7.2 Schedule() prima e dopo il context switch

La funzione chiama prev e next rispettivamente il task che deve essere sostituito e quello che eseguirà successivamente. Essa esegue alcune operazioni fondamentali prima del context switch:

- Calcola il tempo che il processo sottrae al proprio time slice;
- Verifica se il processo prev sia un processo che sta per essere terminato;
- Verifica l'esistenza di processi runnable
  - Se non ce ne sono, next viene settato sullo swapper e attivi e scaduti vengono scambiati;
  - Se ce ne sono, verifica se appartiene agli attivi oppure agli scaduti.
- Determina qual è il processo della run queue che effettivamente ha priorità maggiore;
- Associa alla variabile locale next il puntatore al descrittore del processo che sostituirà prev;
- Invoca la funzione `context_switch()` che esegue lo scambio di contesto

tra i due processi.

La funzione `finish_task_switch(prev)` controlla se il processo passato come parametro è uno zombie, in caso positivo avvia le procedure per eliminarlo. Le ultime istruzioni controllano se altri processi hanno posto il flag di quello corrente a `TIF_NEED_RESCHED`; in tal caso la funzione viene eseguita da capo, altrimenti termina

7.8 Gestione dei Page Frame

Linux opta per Page Frame di dimensioni di 4Kb, per 2 motivi:

- I Page Fault vengono gestiti in modo più efficiente;
- Alcune operazioni risultano più efficienti quando si utilizza un page frame di piccole dimensioni

7.9 Page Descriptor

Il kernel tiene traccia delle informazioni relative ad un page frame, incapsulandole all'interno di un **page descriptor**. Ciascun descrittore ha una dimensione di 32 byte ed è contenuto all'interno di un array chiamato `mem_map`.

Data la dimensione limitata, lo spazio occupato dall'insieme dei descrittori occupa meno dell'1% della memoria disponibile del sistema.

Dato un indirizzo lineare addr è possibile recuperare il page descriptor che gli corrisponde attraverso la macro `virt_to_page(addr)`

Dato l'indirizzo di un page frame pfn è possibile recuperare l'indirizzo del suo page descriptor mediante la macro `pfn_to_page(pfn)`

7.9.1 Campi di un Page Descriptor

Rivesto una certa importanza:

- `_count` → contatore degli utilizzi del page frame
  - Impostato a -1 indica che il page frame è libero;
  - Con valore ≥ 0 indica il numero di processi che lo stanno utilizzando;
- `flags` → descrive lo stato del page frame.

7.10 NUMA

In alcune architetture non tutte le CPU accedono a tutte le zone di memoria con la stessa velocità.

La memoria è divisa in nodi ed il kernel alloca per ogni CPU, quei nodi ai quali essa può accedere con maggiore velocità, riducendo gli accessi ai nodi più lenti.

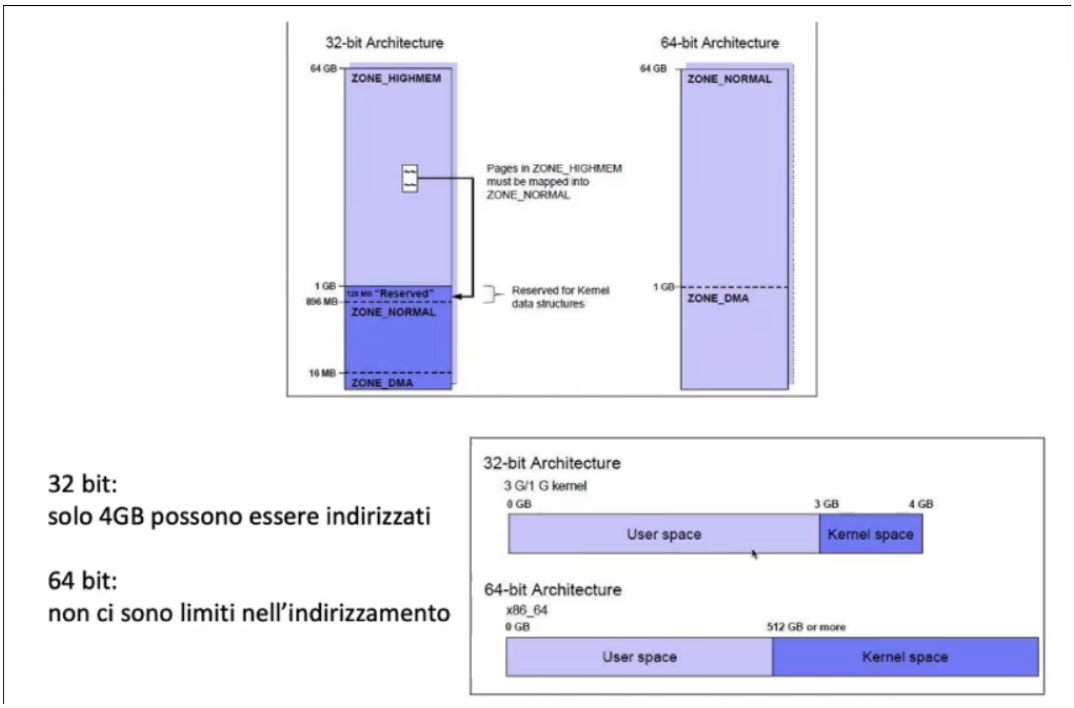
La memoria contenuta all'interno di ciascun nodo è poi suddivisa in zone.

Linux considera la memoria come contenuta in un unico modo → questo gli permette di mantenere la compatibilità con i sistemi che ulitizzano uno schema NUMA (Non Uniform Memory Access)

7.11 Zone di memoria

Linux gestisce la memoria in 3 diverse regioni:

- `ZONE_DMA`;
- `ZONE_NORMAL`;
- `ZONE_HIGHMEM`;



Linux adotta 2 diversi metodi per rispondere alle richieste di allocazione di memoria:

- Se c'è memoria sufficiente le pagine richieste vengono allocate direttamente;
- Se le pagine non possono essere allocate a causa di scarsa disponibilità di memoria, viene attivata una procedura di *page reclaiming*.

Il secondo approccio non è fattibile per quei KCP (kernel control path) che non possono eseguire operazioni bloccanti. In questo caso inviano una atomic request, e nel caso in cui questa fallisca il KCP termina con un errore. Al fine di limitare questi casi il kernel si riserva un numero limitato di pagine per gestire le atomic request.

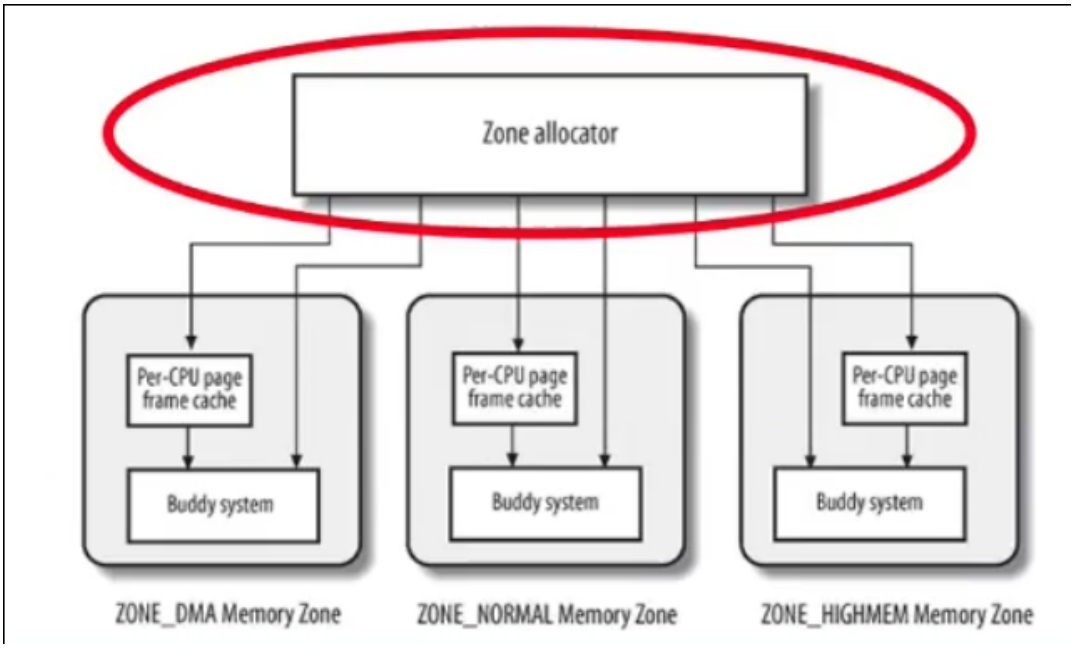
$$reserved\ pool\ size = [\ \sqrt{16\ x\ directly\ mapped\ memory}\ ]$$

7.12 Zoned Page Frame Allocator

Il sottosistema del kernel che gestisce le richieste di allocazione di gruppi di page frame contigui, prende il nome di zoned page frame allocator.

All'interno di ciascuna zona, i page frame vengono gestiti da un componente che prende il nome di Buddy System.

Un limitato numero di pagine viene riservato in alcune cache, che prendono il nome di *Per-CPU page frame cache*



Lo Zone allocator riceve le richieste di allocazione e deallocazione di memoria. Per allocazione lo ZA ricerca una zona di memoria che comprende un numero di pagine necessario per soddisfare la richiesta. All'interno di ogni zona i page frame sono gestiti dal Buddy System.

Lo ZA è il frontend del *kernel page frame allocator*. Le richieste di allocazione vengono sempre ricondotte a questo componente, che ha il compito di vigilare su diversi aspetti:

- I page frame allocati non possono essere presi dal pool di pagine riservate dal kernel per le allocazioni atomiche;
- Quando non c'è disponibilità di pagine, invoca la procedura di page reclaiming;
- Ottimizza i page frame allocati nella `ZONE_DMA`, viste le sue dimensioni limitate.

Il kernel deve quindi stabilire una robusta strategia ed efficiente per allocare gruppi di page frame contigui. Per fare ciò, deve affrontare il problema della *frammentazione esterna*, che viene risolto dal Buddy system

7.12.1 Per-CPU page frame cache

Il kernel spesso rilascia e richiede singole pagine. Per migliorare le prestazioni, ogni zona di memoria definisce una *per-CPU page frame cache*, la quale include alcune pagine di memoria preallocate per essere usate per singole richieste di memoria da parte della CPU locale.

7.13 Kernel Mapping di page frame

La memoria indirizzata direttamente dal Kernel, si limita ai primi 896 Mb. I page frame oltre gli 896 Mb vengono gestiti in modo indiretto.

La allocazione di questi page frame è gestita dalle funzioni `alloc_page()` e `alloc_pages()`. Esse non restituiscono l'indirizzo lineare del page frame, ma l'indirizzo lineare del corrispondente page descriptor, il quale è sempre conservato nella memoria bassa.

Gli ultimi 128 Mb nello spazio degli indirizzi lineari del kernel è riservato alla gestione di questi page frame, la quale viene effettuata con 3 diversi meccanismi:

- Permanent Kernel Mapping;
- Temporary Kernel Mapping;
- Non Contiguous Memory Allocation;

7.14 Il problema della frammentazione esterna

Linux adotta un sistema dedicato alla gestione dei page frame al fine di risolvere il problema della frammentazione esterna. Si possono adottare 2 filosofie diverse per far fronte a questo aspetto:

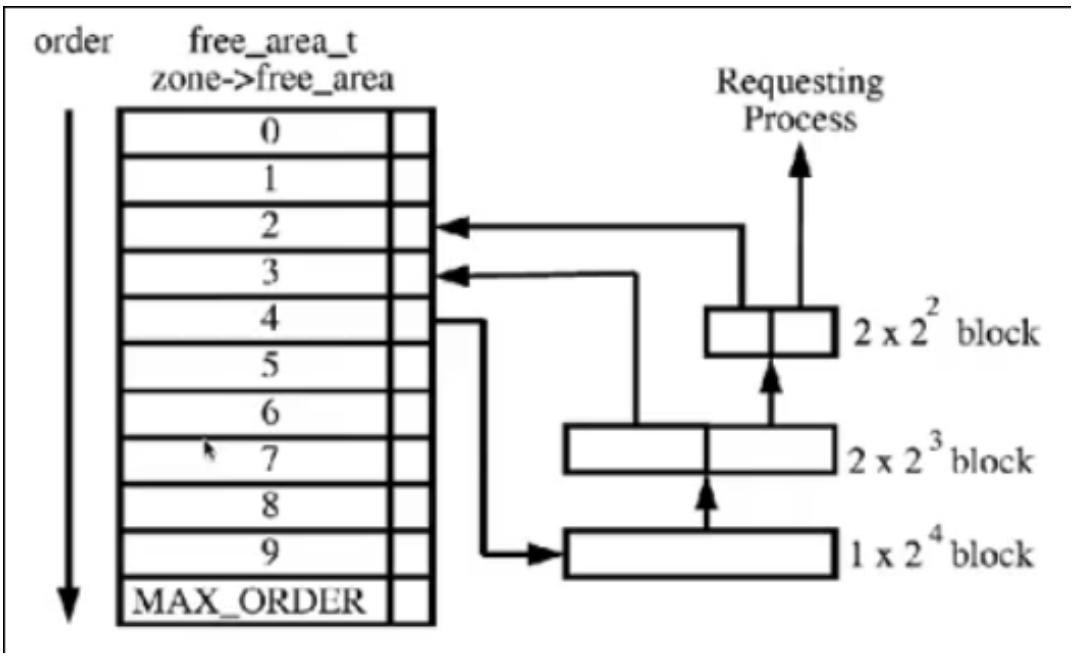
- Mappare intervalli di indirizzi lineari su zone di memoria non contigue;
- Implementare un componente dedicato, che tenga traccia delle regioni contigue disponibili in memoria centrale.

Linux opta per la seconda strategia, proprio per alcuni vantaggi che questa offre:

- La memoria riserva in alcuni casi deve essere continua;
- Riduce gli aggiornamenti delle Page Table;
- Permette la gestione di grandi aree di memoria con pagine di 4Mb

7.15 Il Buddy System

I page frame in memoria sono organizzati in 11 liste $k = 0,1,\ldots, 10$, che contengono gruppi di 2^k pagine. Dato un gruppo di ordine k, l'indirizzo fisico del primo page frame di tale gruppo è un multiplo della dimensione della pagina (4 kb = 4096 byte = 2^{12} byte)



7.15.1 Filosofia del Buddy System

Quando un processo necessita di un dato numero di page frame, fisssa un ordine k e sottopone al buddy system una richiesta.

Se esso è in grado di evaderla, restituisce un indirizzo fisico dei page frame allocati, altrimenti scorre le liste degli ordini superiori, e divide un gruppi più grande in gruppi più piccoli.

In caso contrario, incrementa k di 1 ed effettua nuovamente il controllo. Se è disponibile un gruppo nella lista di ordine *k+1*:

- Dimezza il gruppo in 2 sottogruppi di ordine k;
- Alloca un gruppo, e inserisce l'altro nella lista di ordine k.

In caso contrario incrementa nuovamente k;

Supponiamo trovi un gruppo di ordine $k+a$, con `a>1`:

- Partiziona il gruppo in due sottoruppi di dimensione `k+a-1`;
- Inserisce il primo sottogruppo nella lista di ordine `k+a-1`;
- Suddivide ulteriormente il secondo sottogruppo in due di dimensioni `k+a-2`;
- Dopo a iterazioni di questo processo, arrivato all'ordine k, alloca un gruppo e accoda l'altro.

7.15.2 Strutture dati

Linux considera la memoria del sistema divisa in 3 zone, pertanto utilizza un diverso buddy system per ciascuna di esse.

Ciascuno dei 3 buddy system fa riferimento a delle strutture dati:

- array `mem_map` → ciascuna delle 3 zone di memoria è rappresentata da una porzione di questo array. Ciascuna zona è caratterizzata da uno zone descriptor che ne indica il primo elemento ed il numero di elementi;
- un array di tipo `free_area`, conservato nel campo `free_area` dello zone descriptor.
  - Questo array ha 11 posizione, una per ciascun ordine.

Il `k-esimo` elemento del secondo array rappresenta la testa di una lista doppiamente linkata che punta al primo page frame del primo gruppo di page frame di ordine k

7.15.3 Liberazione di un blocco

La funzione incaricata di liberare i blocchi per un buddy system è `_free_page_bulk()`.

Essa opera con gli interrupt disabilitati e dopo che un opportuno spin lock è stato attivato. Dopo aver liberato un blocco la funzione controlla se è possibile fondere il gruppo di pae frame appena liberato con il suo gemello, verificando 3 condizioni:

- Entrambi i blocchi devono avere la stessa dimensione b (lo stesso ordine k);
- Gli indirizzi dei due blocchi si riferiscono a regioni di memoria contigue;
- L'indirizzo fisico del primo page frame del primo blocco è un multiplo di 2 x b x 2^{12}

8 Lezione del 29-03

8.1 Per-CPU Page Frame Cache

Molto spesso il kernel richiede una singola pagaina, e far riferimento al buddy system, per allocazioni di questa taglia crea un inutile overhead. Il kernel quindi riserva un pool di page frame come cache.

Il kernel considera 2 tipi di cache di page frame:

- **hot cache** → pagine che vengono usate da un processo subito dopo essere state richieste;
- **cold cache** → possono essere utilizzate anche non immediatamente dopo l'allocazioni (utilizzate spesso dalle DMA);

La struttura dati che le gestisce è un array di strutture dati di tipo `per_cpu_pageset`.

Anche questo array è contenuto nel memory zone descriptor

8.2 La funzione `__alloc_pages()`

Riserva gruppi di page frames in una specifica zona di memoria

Signature = `_alloc_pages(gfp, order, zonelist)`

Esso si serve inoltre di una funzione `zone_watermark_ok()`, per calcolare una soglia minima (*mark*), in base alla quale stabilire se i page frame richiesti possono essere allocati o meno; Restituisce 1 se il valore è maggiore della soglia minima

Quando la funzione `alloc_pages()`, si effettuano una serie di step prima di restituire un indizzo valido o NULL, a seconda della disponibilità di memoria nel sistema:

- Effettua un primo scan della zona di memoria;
- Se la funzione non termina durante il primo scan, non c'è abbastanza memoria;
  - Quindi sveglia i kernel thread kswapd per effettuare un page reclaiming;
- Abbassa la soglia mark ed effettua un secondo scan;
- Se nemmeno in questo caso la funzione termina prima della fine dello scan, la memoria disponibile è insufficiente e se il KCP è autorizzato fa ricorso alle pagine della `low-on-memory reserver`, terminando con un NULL (in caso di insuccesso);
- Se i passi precedenti hanno liberato delle pagine, viene effettuato un terzo scan;
- Se lo scan fallisce e il processo può essere messo in sleep, lo sospende
  - Se il processo non può essere sospeso, cerca un processo da uccidere, con la funzione `out_of_memory()`.

8.3 La funzione `__free_pages()`

Riceve in input:

- il page descriptor del primo page frame del gruppo;
- L'ordine del gruppo di page frame.

La funzione:

- verifica che il page descriptor faccia riferimento ad una regione della memoria dinamica;

- Decrementa il contatore page → `_count` (Se questo è maggiore di 0 termina);
- Se l'ordine k è 0, si tratta di un page frame della hot cache, per cui chiama la funzione `free_hot_page()`;
- Se l'ordine k è maggiore di 0, rilascia il gruppo al buddy system di competenza della zona di memoria cui il gruppo appartiene.

8.4 Memory Area Management

Lo spazio all'interno delle pagine potrebbe essere parzialmente inutilizzato dando origine ad una frammentazione interna

Alcune versioni di Linux precedenti la 2.6, contenevano tale fenomeno permettendo allocazioni di blocchi la cui dimensione fosse potenza di 2 (distribuzione geometrica). In questo modo la frammentazione interna è sempre minore del 50%.

Linux 2.6 adotta un sistema simile a quello implementato dal sistema Solaris2.4, implementando uno Slab Allocator

8.5 Lo Slab Allocator nel dettaglio

Lo Slab Allocator tratta le aree di memoria come degli oggetti che vengono allocati e deallocati all'interno di contenitori detti slab (lastre).

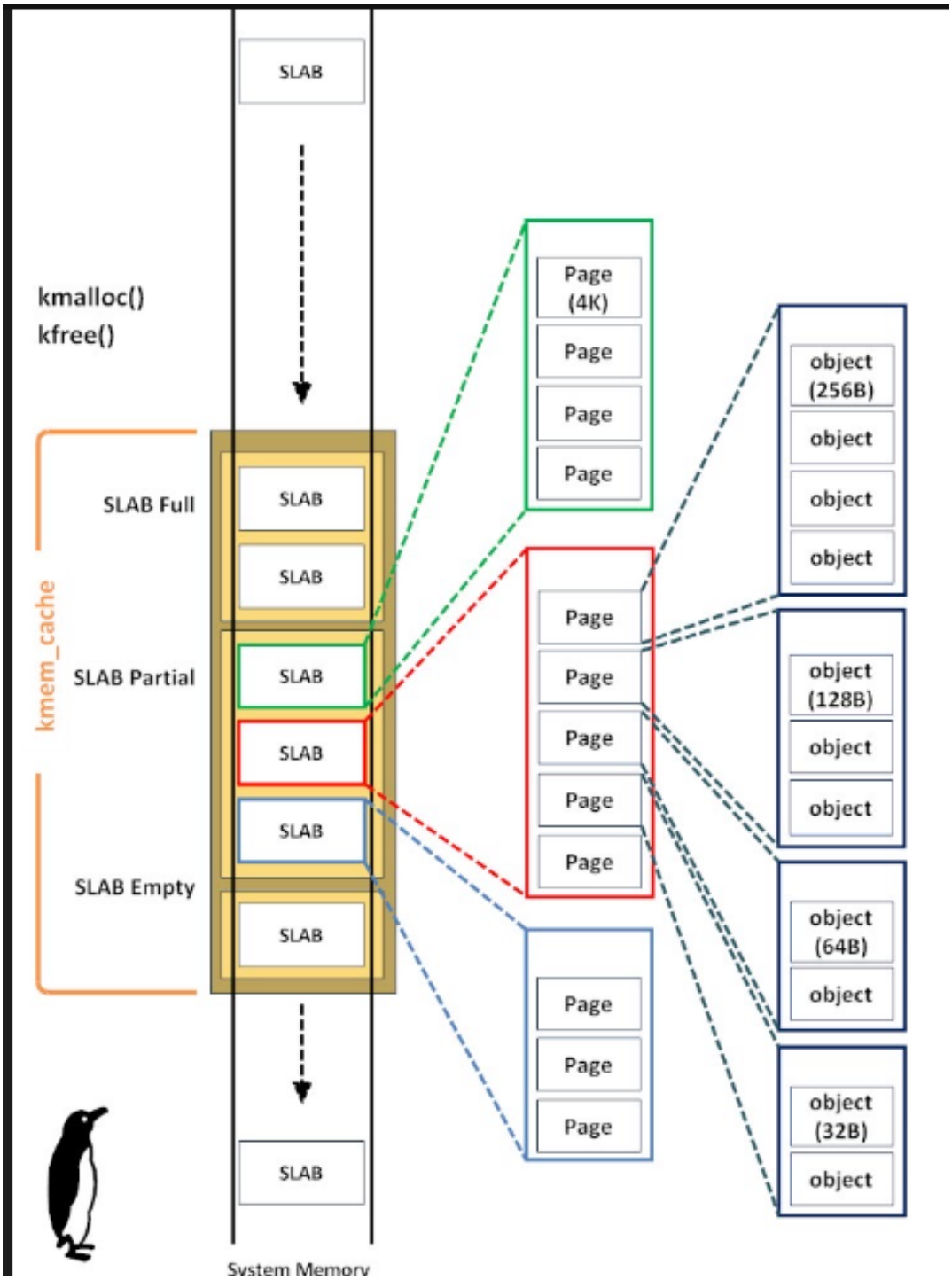
Esso nasce da alcune considerazioni fondamentali:

- Sulla base delle strutture dati da memorizzare in una zona di memoria è possibile ottimizzare il processo di allocazione e deallocazione
  - Le aree di memoria sono viste come oggetti, dotate di un costruttore e di un distruttore.
  - Gli oggetti deallocati non sono immediatamente distrutti
- Le funzioni del kernel richiedono con elevata frequenza l'allocazione di alcune strutture dati (es. creazione di un processo). Oggetti allocati e deallocati possono essere riutilizzati
- Le allocazioni possono essere gestite in base alla frequenza. Quelle molto frequenti con l'allocazione di oggetti, quelle meno frequenti con la distribuzione geometrica delle dimensioni delle aree di memoria.
- Gli indirizzi delle zone di memoria non si concentrano necessariamente intorno a multipli di potenze di 2;
- Il buddy system (hot cache) induce una alterazione dei contenuti delle cache HW (footprint).

Lo Slab Allocator, gestisce delle regioni di memoria, dette Cache. All'interno di ciascuna di queste sono allocate un certo numero di lastre. A loro volta ciascuna di queste contiene un certo numero di oggetti.

Le Cache e le Slab sono caratterizzate rispettivamente da un Cache Descriptor e da uno Slab Descriptor.

Periodicamente il kernel fa uno scan delle cache e rilascia i page frames che corrispondono a lastre vuote



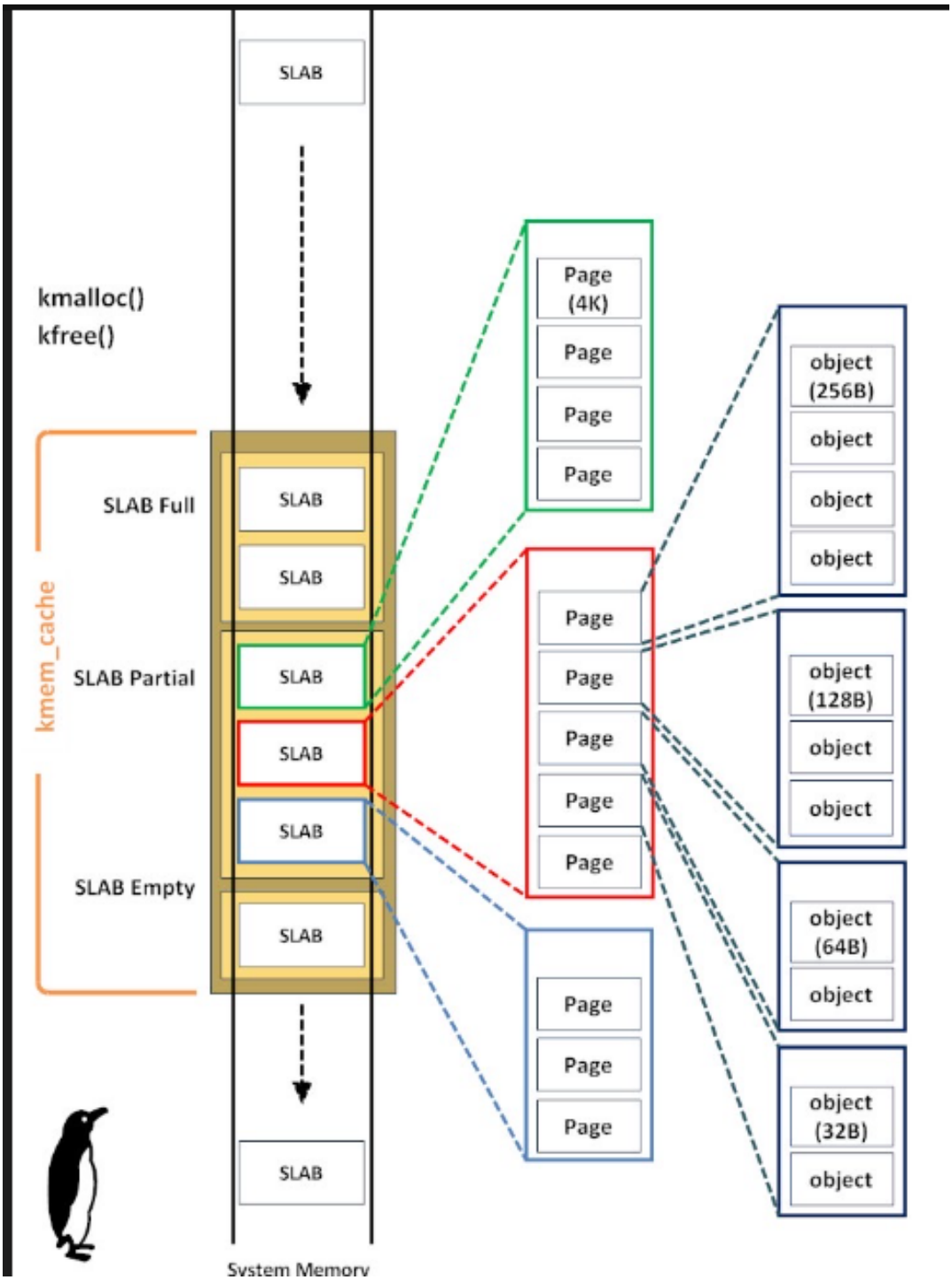
8.6 Cache Generiche e Specifiche

Divise in 2 categorie:

- Generiche → utilizzate solo dallo Slab Allocator, per i propri scopi

  - ad es per contenere i cache descriptor (`kmem_cache`);

- Specifiche → contenono slab con oggetti tutti dello stesso tipo
  - ad es l'oggetto per i file aperti, o per le page table dei processi



8.7 Allocazione e Deallocazione di Slab

Quando lo Slab allocator deve allocare una nuova slab esegue in cascata una serie di step:

- Richiede allo *Zoned Page Frame Allocator* un blocco di page frame da associare alla slab, specificando quale cache necessita della memoria;
- Associa la nuova slab alla cache, se e solo se
  - È stata inoltrata una richiesta di allocazione per un nuovo oggetto;
  - Nella cache non è già presente un oggetto libero.

Poichè il kernell tiene traccia di come viene utilizzata la memoria, deve:

- Poter determinare efficientemente se dei page frame sono utilizzati dallo slab allocator;
- Ottenere in modo efficiente il corrispondente cache descriptor e slab descriptor.

Le informazioni necessarie sono inserite nel campo Lru del page descriptor

Lo Slab Allocator libera una slab in 2 casi:

- Al suo interno ci sono troppi oggetti liberi;
- La slab è completamente inutilizzata (viene effettuato un check periodico)

Se gli oggetti di una slab sono forniti di un metodo distruttore, questo viene invocato per tutti gli oggetti al suo interno e i page frame corrispondenti alla slab vengono liberati.

8.8 Gli Object Descriptor

Ciascun oggetto contenuto in una slab è dotato di un Object Descriptor.

Un Object Descriptor è un valore intero. Ha senso se l'oggetto è libero, in quanto indica la posizione del successivo oggetto libero, implementando una sorta di lista linkata. Come gli altri descrittori può trovarsi:

- Fuori dalla slab, ossia nella cache generica puntata dal cache descriptor in cui l'oggetto si trova;
- Dentro la slab, immediatamente prima dell'oggetto che descrive

8.9 Slab Coloring

GLi oggetti gestiti dallo slab allocator sono allineati in memoria, ossia il loro indirizzo è multiplo di una potenza di 2 (aln). Solitamente si sceglie la lunghezza di una parola macchina, quindi 4 in un sistema a 32 bit.

Quando gli oggetti vengono associati a linee di cache HW finiscono, con molta probabilità, sulle stesse linee di cache. A ciascuna slab è associato un colore *col* e tutti i suoi oggetti vengono allineati a partire dall'indirizzo `col x aln + dsize`, dove *dsize* rappresenta la somma delle dimensioni della slab e di tutti i suoi oggetti.

8.10 Local Cache e Memory Pool

Il kernel riserva in anticipo alcune risorse, tra cui:

- Local Cache → cache locale di slab per ciascuna CPU, il sistema ricorre allo slab allocator solo quando non riesce a soddisfare le richieste sulla base di queste risorse.
- Memory Pool → insieme di page frame riservati a priori per uno specifico componente del kernel
  - Vengono utilizzate solo quando il sistema è sovraccarico e la memoria è insufficiente.

Si tratta di un insieme diverso dal reserved pool, che può essere utilizzato solo per soddisfare le atomic memory allocation request

8.11 Processi utente e gestione della memoria

Il Kernel gestisce la memoria dinamica facendo ricorso a diverse funzioni.

Le strategie adottate dal kernel, garantiscono una gestione efficiente delle risorse di memoria, per 2 ragioni principali:

- Il Kernel ha sempre la priorità massima;
- Gode della massima fiducia (si suppone sia bug free e non implementi azioni malevoli).

La gestione della memoria per conto dei processi funziona in maniera diversa:

- Le richieste di memoria dei processi utente sono considerati non urgenti (ritardati il più possibile);
- Il kernel deve verificare il corretto comportamento dei processi utente.

8.12 Lo spazio degli indirizzi

Lo spazio degli indirizzi di un processo è l'insieme di tutti gli indirizzi lineari che il processo può utilizzare.

Ciascun processo può gestire un limitato sottoinsieme di tutti gli indirizzi lineari disponibili.

Il kernel può aggiungere o eliminare dinamicamente un intervallo di indirizzi lineari, per diverse ragioni:

- L'esecuzione di un comando da shell, genera un nuovo processo;

- Un processo invoca una *exec()* per eseguire un programma differente;
- Un processo crea una ragione di memria condivisa;
- Un processo chiama una delle funzioni *malloc(), calloc(), free()*.

8.13 Regioni di memoria

Gli intervalli di indirizzi lineari sono rappresentati dal kernel mediante regioni di memoria. Ciascuna regione di memoria è caratterizzata da:

- Il primo indirizzo lineare in essa contenuto (multiplo di 4096);
- La sua lunghezza (multiplo di 4096);
- Dei diritti di accesso.

Per il kernel è fondamentale conoscere il proprietario di ciascuna regione di memoria al fine di poter gestire correttamente i page fault:

- Alcuni sono causati da errori del programma;
- Altri sono causati dal fatto che una pagina richiesta non sia in memoria.

Ciascuna regione di memoria è caratterizzata da un memory descriptor, il cui riferimento *mm* è nel process descriptor del processo, che ne risulta proprietario.

Le regioni di memoria, che appartengono ad uno stesso processo non si possono sovrapporre.

Quando una nuova regione di memoria viene allocata o deallocata, in modo adiacente ad una già esistente per lo stesso processo, il kernel verifica:

1. Se è possibile allargare una regione esistente;
2. Se è necessario creare una nuova regione;
3. Se è possibile ridimensionare le regioni di memoria rimanenti;
4. Se è necessario dividere la regione in due sotto-regioni.

8.14 Strutture dati

Il memory descriptor possiede un campo `map_count` che indica il numero di regioni appartenenti al processo ed un puntatore mmap ad una lista concatenata

Tutte le regioni appartenenti allo stesso processo sono organizzate in una lista concatenata.

Le regioni nella lista compaiono in ordine crescente rispetto all'indirizzo lineare di inizio. Fra una regione e l'altra possono esserci regioni allocate a processi differenti

8.15 Ricerca di una regione di memoria

Una delle operazioni più frequenti per il kernel è la ricerca della regione che contiene un determinato indirizzo.

Una delle strategie adottate dal kernel per la ricerca delle regioni è la ricerca lineare scorrendo la lista. Il kernel comincia dalla testa e arresta la ricerca appena trova una regione che termina dopo l'indirizzo cercato.

Questo approccio è efficiente solo quando un processo ha un numero limitato di regioni di memoria.

Linux 2.6 adotta anche un approccio basato su alberi `red-black`, la cui radice è puntata dal campo `mm_rb` del memory descriptor

8.15.1 Alberi red-black

Alberi binari di ricerca estesi. La caratterizzazione degli alberi red-black avviene attraverso la formulazione di 4 proprietà vincolanti.

1. Ogni nodo dell'albero è *rosso* o *nero*;
2. OGni foglia dell'albero (NIL) è nera;
3. Se un nodo è rosso entrambi i suoi figli sono neri;
4. Dato un nodo, tutti i percorsi discendenti che raggiungono una foglia contengono lo stesso numero di nodi neri

La ricerca di una regione attraverso l'albero ha sempre un costo pari a $O(log(n+1))$ dove n è il numero di regioni. Le operazioni di aggiornamento dell'albero hanno costo logaritmico.

Aggiungere un nuovo livello nell'albero significa aumentare di una iterazione il costo dell'algoritmo di ricerca.

8.16 Linux e la ricerca di una regione di memoria

Adotta entrambe le strutture dati per gestire le regioni di memoria.

La lista e l'albero utilizzano puntatori distinti, che puntano alle stesse regioni di memoria.

Generalmente, la lista è utilizzata per scorrere tutto lo spazio degli indirizzi, mentre l'albero red-black è usato per cercare una regione che contenga un determinato indirizzo.

8.17 Diritti di accesso ad una regione di memoria

Una regione di memoria consiste in un insieme di pagine contigue.

Ciascuna pagina è accompagnata da *flag* che ne caratterizzano l'utilizzo:

- Read, Write, Present, User, Supervisor nella Page Table entry;
- Locked, Dirty, LRU, SLAB nel page descriptor.

A questi si aggiunge un'ulteriore serie di flag espressa dal campo `vm_flags` nel memory region object (di tipo `vm_area_struct`)

8.18 Demand Paging

Tecnica di allocazione dinamica della memoria che tenta di ritardare quanto più possibile la reale assegnazione delle pagine fisiche.

Questo ha senso, poichè un processo non utilizza mai tutte le pagine richieste sin dall'inizio della sua esecuzione; inoltre, ci sono pagine che non vengono mai utilizzate.

Questa tecnica offre 2 principalei vantaggi:

- Mantenere un più alto numero di pagine disponibili;
- Aumentare il throughput del sistema, svolgendo lo stesso lavoro con un minor numero di pagine di memoria.

Il principale svantaggio è che i page fault vanno gestiti e producono overhead. Tuttavia, per il principio di località, i processi tendono ad utilizzare sempre un ristretto gruppo di pagine, rendendo i page fault abbastanza rari.

8.19 Page Reclaiming

Linux limita al massimo i controlli sulla quantità di memoria che può essere allocata ad un processo o ad un kernel thread, così da permetterne un uso quanto più possibile efficiente.

Il modo, in cui la RAM viene gestita dipende dal carico del sistema:

- *Basso carico* → molte pagine sono utilizzate per tenere in memoria informazioni caricate dal disco;
- *Elevato carico* → le pagine di memoria vengono principalmente utilizzate per le cache dei processi e del kernel

Le pagine allocate ad una cache in memoria, non vengono gestite in modo diretto, poichè non è dato sapere quando un processo non utilizzerà più i dati in tale cache.

Linux implementa un meccanismo di page reclaiming, che gli permette di recuperare le pagine, quando se ne crea la necessità.

8.20 Selezione di una pagina target

Linux seleziona le pagine da liberare attraverso un Page Frame Reclaiming Algorithm (PFRA).

Le pagine che rappresentano un possibile candidato per il PFRA devono essere pagine non libere, ossia non devono appartenere già al pool di pagine disponibili per il Buddy System.

Al fine di selezionare una eventuale pagina target, il PFRA distingue le pagine allocate in 4 diverse categorie:

- Non reclamabili → pagine che non possono essere liberate per diverse ragioni
  - Pagine libere;
  - Pagine riservate;
  - Pagine allocate direttamente dal kernel;
  - Pagine locked;
- Swappabili → pagine il cui contenuto deve essere scritto sul disco prima che esse vengano liberate;
- Sincronizzabili → pagine che fanno da cache per dati su disco o che rappresentano un buffer per qualche device. Esse devono essere allineate con il contenuto di cui sono cache, prima di essere liberate;
- Scartabili → pagine non utilizzate, come quelle allocate per oggetti vuoti nelle lastre dello slab allocator.

Un aspetto importante, di cui il PFRA deve tenere conto quando seleziona una pagina, è se essa è condivisa o meno

8.20.1 Principi alla base di un PFRA

La reale difficoltà consiste nel progettare un algoritmo che sia in grado di funzionare bene anche in contesti molto diversi fra loro:

- Sistemi desktop → le richieste di memoria sono poco frequenti, mentre la reattività del sistema è fondamentale;
- Sistemi per workstation → la gestione delle richieste di memoria è fondamentale, come nel caso dei grandi server di basi di dati.

Il supporto teorico alla soluzione del problema è scarso e la progettazione di un buon PFRA è un task basato prettamente su conoscenze empiriche.

La diretta conseguenza è che la parte di codice che implementa questo componente varia molto velocemente nel tempo.

IL PFRA di Linux si basa su 4 scelte euristiche fondamentali:

- Seleziona prima le pagine *harmless*
  - Si tratta di quelle pagine in qualche cache e che non sono referenziate da alcun processo. Esse vanno selezionate prima delle pagine associate allo spazio di indirizzi di qualche processo;

- Rendi tutte le pagine di un processo in User Mode reclamabili
  - Eccezion fatta per le pagine locked, tutte le pagine di un processo sono potenzialmente soggette a reclaiming. Se un processo è in sleep per lungo tempo, potrebbe perdere tutte le sue pagine;
- Reclama le pagine condivise dopo aver eliminato i collegamenti nelle page table;
- Reclama solo le pagine non utilizzate (sfrutta il LRU)

8.21 Periodic Reclaiming

Il PFRA esegue periodicamente un reclaiming, attraverso 2 diversi meccanismi:

- Il kernel thread kswapd
  - Necessario poichè alcune richieste di allocazione sono effettuate da interrupt e deferrable function, che non permettono la sospensione del processo
  - I kernel thread possono sfruttare cicli di clock, in cui la CPU sarebbe idle;

Quando la funzione `__alloc_pages()` si rende conto che le pagine di memoria sono sotto una soglia critica, sveglia il thread per il corrispondente nodo di memoria.

La funzione `cache_reap()` → questa funzione libera pagine allocate a oggetti liberi nelle lastre dello slab allocator. Ogni volta che viene eseguita, la funzione richiede allo scheduler di pianificare la sua successiva esecuzione

9 Lezione del 31-03

9.1 Virtual File System (VFS)

I sistemi Linux presentano la capacità di coesistere e interagire con altri sistemi presenti sulla stessa macchina.

Un esempio concreto di tale capacità è dato dalla possibilitù di montare dischi o partizioni, che presentano formati di file differenti.

Ciò è possibile attraverso un SW layer del kernel che prende il nome di Virtual File System che gestisce l'intero insieme di system call tipico dei fiesystem standard UNIX.

Il VFS lavora come livello di astrazioni fra i programmi di applicazione e la reale implementazione di un filesystem

Divisi in 3 categorie:

- Disk-based filesystem → gestiscono le informazioni memorizzate in dischi locali o in dispositivi, il cui comportamento è assimilabile a quello di un disco locale;
- Network filesystem → permettono l'accesso alle informazioni memorizzate su pc collegati in rete;
- Special filesystem → non gestiscono dati realmente memorizzati su un disco locale o remoto;

9.1.1 Common File Model

Rappresenta e modella tutti i filesystem supportati. Nel CFM tutte le entità coinvolte sono rappresentate attraverso file.

Dato che Linux non implementa i filesystem nel loro formato nativo, deve ricostruire tali oggetti in memoria "on the fly". Si può guardare al CFM come ad un sistema ad oggetti, caratterizzati da dati, che conservano le informazioni e metodi che servono a manipolarle.

Un CFM consiste in:

- Il **superblocco** → contiene informazioni relative al filesystem che è stato montato;
- L' **inode** → contiene informazioni relative ad un file specifico
  - Ciascun oggetto di tipo inode è associato ad un inode number, che identifica in maniera univoca il file nel filesystem;
- Il **file object** → contiene informazioni sul tipo ed il grado di interazione fra un processo ed un file aperto
  - Queste informazioni sono tenute solo in memoria centrale e non trovano alcuna corrispondenza con alcun elemento sul disco;
- Il **dentry object** → contiene le informazioni che associano ciascuna entry della directory con il file corrispondente
  - Generalmente ogni filesystem ha un modo peculiare di memoriazzare questa associazione.

9.1.2 Strutture dati del VFS

Il VFS è contenuto in un oggetto, implementato da una struttura dati `super_block`, la quaale contiene una serie di campi dati ed un puntatore ad una tabella di funzioni, che rappresentano i metodi dell'oggetto.

9.1.3 L'oggetto superblock

Tutti gli oggetti superblock sono collegati tra loro in una lista doppiamente concatenata la cui testta è contenuta nella variabile `super_blocks`, mentre il puntatore `s_list` contiene l'indirizzo del successore.

Il campo `s_fs_info` punta ad una struttura dati contenente le informazioni sul tipo di filesystem, cui il superblock fa riferimento.

Per questioni di efficienza, Linux mantiene e aggiorna una copia dell'oggetto superblock in memoria, impostando un flag `s_dirt` per assicurarne la sincronizzazione con la copia su disco

9.1.4 Operazioni sul superblock

La struttura `super_block` contiene un campo `s_op`, che punta ad una struttura `super_operations` relativa ai metodi implementati per tale oggetto.

`alloc_inode()`
`destroy_inode()`
`read_inode()`
`dirty_inode()`
`write_inode()`
`delete_inode()`
`...`
`quota_read()`
`quota_write()`

Queste operazioni sono disponibili per qualunque file system. Se i metodi non sono implementati, i corrispondenti puntatori sono NULL.

9.1.5 L'oggetto inode

Tutte le informazioni di cui il filesystem ha bisogno di gestire un file, sonon contenute nella struttura inode. Sebbene il nome di un file possa cambiare, l'inode che gli è stato associato rimane sempre lo stesso.

Linux mantiene una copia di un iniode in memoria e usa un flag `i_state` per capire se è necessario sincronizzarne il contenuto con la sua copia su disco.

Il campo `i_state` fornisce anche informazioni quali:

- `I_LOCK` (operazioni di `I/O`);
- `I_FREEING` (rilasciato);
- `I_CLEAR` (contenuto non più significativo);
- `I_NEW` (allocato e non ancora utilizzato);

9.1.6 Il File Object

Descrive il modo in cui un processo interagisce con un file che ha aperto. L'oggetto viene creato all'apertura del file e consiste in una struttura, che non ha un corrispondente su disco, ed è quindi priva di un flag **dirty**.

L'informazione più importante memorizzata all'interno della struttura è il **file pointer**. Esso è conservato qui e non nell'inode, poichè più processi potrebbero accedere allo stesso file.

I file object sono allocati in oggetti detti **filp**, conservati insieme al loro descrittore, in una delle cache dello slab allocator

9.1.7 Dentry File

Quando una directory viene letta in memoria, il VFS la trasforma in un oggetto dentry, implementato dalla struttura dentry. Il kernel scompone un path nelle sue componenti e crea un oggetto dentry per ciascuna di esse. Questo oggetto associa ciascuna parte al corrispondente inode.

9.1.8 Tipi di filesystem

Linux offre la possibilità di gestire diversi tipi di filesystem, di conseguenza il kernl deve avere a disposizione tutte le informazioni necessarie a poterli gestire.

Il kernel rende possibile questa opzione attraverso una procedura, che va sotto il nome di **Filesystem Type Registration**.

Ciascun filesystem è caratterizzato da una struttura dati di tipo `file_system_type`. Tutti i filesystem object sono contenuti in una lista concatenata.

9.1.9 La gestione dei filesystem

All'avvio del sistema, la funzione `register_filesystem()` inserisce un oggetto di tipo `file_system_type` per ciascun tipo di filesystem disponibile all'interno della lista dei tipi di filesystem.

Questa funzione viene invocata anche quando il kernel carica un modulo che implementa un filesystem.

Inoltre è disponibile una funzione `get_fs_type()`, che prende in input il nome di un filesystem e restituisce il corrispondente `file_system_type` object.

Il Filesystem principale (root) è il primo ad essere caricato ed è quello in cui sono presenti i programmi e gli script necessari all'avvio del sistema.

I filesystem possono essere montati su una directory di un filesystem già montato, che viene considerata come **mounting point** e diviene la root del filesystem montato.

Il nuovo filesystem viene considerato come **child** del filesystem sul quale è stato montato.

La root directory di un filesystem appena montato, nasconde il contenuto della directory usata come mount point.

9.1.10 I namespace

I filesystem sono organizzati come un albero e nei sistemi Unix tradizionali esiste sempre un solo albero.

Linux, si differenzia in questo, facendo uso dei **namespace**.

Diversi processi possono condividere lo stesso namespace, ossia l'albero dei filesystem correntemente montati. Tuttavia, se un processo monta un nuovo filesystem, questo rimane visibile solo a questo, modificando solo il namespace del processo che lo ha montato.

Il namespace è implementato da una struttura dati chiamata namespace, il cui puntatore è un campo del process descriptor

9.1.11 Filesystem Mounting

NEi sistemi Unix tradizionali, ciascun Filesystem può essere montato solo una volta. Linux differisce in questo → è possibile montare lo stesso filesystem più volte, su directory differenti.

Sebbene lo stesso filesystem possa essere acceduto da diversi punti, esso rimane unico, ossia **esiste un solo superblocco** nel sistema che lo identifica.

È possibile montare più filesystem differenti sullo stesso mount point; tuttavia, ciascun nuovo filesystem nasconde il vecchio

10 Lezione del 09-04 - I sistemi operativi Real-Time

Questo tipo di sistema operativo deve garantire che una elaborazione (o task) termini entro un dato vincolo temporale o scadenza (detta in gergo deadline). Per garantire questo è richiesto che la schedulazione delle operazioni sia fattibile. Il concetto di fattibilità di schedulazione è alla base della teoria dei sistemi real-time ed è quello che ci permette di dire se un insieme di task sia eseguibile o meno in funzione dei vincoli temporali dati.

Questo tipo di sistema è implementato via SW.

Il collante tra il livello SW che implementa la logica del sistema di controllo e le risorse HW controllate è il SO.

È presente quindi un livello di astrazione **HAL** (Hardware Abstraction Level).

I task non real-time vengono gestiti dal HAL con una politica BEST EFFORT. Quelli real-time vengono gestiti attraverso lo SCHEDULER che implementa algoritmi di scheduling specifici

10.1 Event Driven

Un SO di questo tipo è in grado di schedulare un task nello stesso istante in cui si verifica l'evento che lo ha attivato

Vantaggi	Svantaggi
Intuitivo e semplice da usare	Complesso da realizzare

10.2 Time Driven

Un SO event-driven è irrealizzabile se l'unità di elaborazione è di tipo digitale e quindi intrinsecamente quantizzato nel tempo

Un approccio realizzabile consiste nel rilevare periodicamente l'occorrenza di eventi e di gestire di conseguenza i relativi task. Tale approccio prende il nome di *Time Driven*. Il periodo di tempo che intercorre tra 2 rilevazioni consecutive è detto **periodo di rilevazione** (o periodo di scansione) *Ts*.

Deve gestire necessariamente le problematiche relative alla gestione sincrona di eventi asincroni (che attivano i task). In un sistema di controllo digitale, un evento può essere associato al valore logico di una variabile binaria (segnale logico).

Vantaggi	Svantaggi
Semplice da realizzare e reattivo	Flessibilità

10.2.1 Problema 1 → Osservabilità degli eventi

Un evento può non essere osservabile. Il segnale logico associato all'evento rimane attivo per un tempo inferiore del periodo di rilevazione.

10.2.2 Problema 2 → Ritardo di rilevazione

Ogni occorrenza di un task è soggetta ad un ritardo di rilevazione che impatta necessariamente sullo start time del task.

10.2.3 Problema 3 → Ordine di occorrenza

L'ordine in cui si presentano 2 o più eventi occorrenti tra due rilevazione succesive viene perso.

10.3 WatchDog

Il So controlla periodicamente che le deadline dei task real-time vengano rispettate attraverso un **WATCHDOG TIMER**. Allo scadere del timer se una deadline è scaduta, l'anomalia è segnalata e viene eseguita una routine di emergenza

10.4 Task periodici e Aperiodici

I task che devono essere eseguiti ogni p unità di tempo sono detti periodici (di periodo p).

Tutti gli altri task sono aperiodici

Task che richiedono il processore in modo impredicibile sono detti **sporadici**

10.5 Deadline

Istante di tempo dopo il qale la computazione non è semplicemente in ritardo, ma errata.

Si tratta di vincoli di tempo stringenti imposti ai task dall'ambiente esterno

10.6 Scheduling Real-Time

- Hard Real-Time → scandenza obbligatoria
  - In caso contrario si potrebbe incorrere in malfunzionamenti;
- Soft Real-Time → scadenza desiderabile ma non obbligatoria;

Gli eventi legati all'esecuzioni di tali task possono essere o periodici o aperiodici.

Ciò implica che un sistema che deve eseguire diversi task periodici è in grado di gestirli solo se i tempo da essi richiesti soddisfano determinati vincoli

Dati **m** task periodici con un periodo `P_i` e richiedenti `C_i` secondi di tempo CPU per essere gestiti, essi possono essere soddisfatti solo se:

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

10.7 Rate Monothonic Scheduler

Scheduler a priorità statica (abbrieviato RMS).

Assegna le priorità ai task in base al loro periodo T, inteso come la quantità di tempo compresa fra l'arrivo di un istanza del task e la successiva.

Il tempo di esecuzione C è la quantità di tempo di elaborazione richiesta da ogni occorrenza del task e in un sistema single-core il tempo di esecuzione non deve essere maggiore del periodo, ossia C < T

Se tutti i processi vengono eseguiti completamente, si può calcolare l'utilizzo del processore come

$$U = \frac{C}{T}$$

10.7.1 Test di schedulabilità

Condizione sufficiente, ma non necessaria data da:

$$\sum_{i=1}^{n} \frac{C_i}{T_i} \leq n(\sqrt[n]{2} - 1)$$

Se il numero di processi n tende ad infinito, questa espressione tende a

$$\lim_{n\to\infty} n(\sqrt[n]{2} - 1) = ln(2)$$

RMS può garantire tutte le scadenze con un utilizzo del processore intorno al 69,3% e quindi il restante 30,7% della CPU può essere utilizzato per la schedulazione di processi non real-time

10.8 EDF Scheduler

Utilizza una coda di priorità e tiene in considerazione le scadenze di ogni task. Ogni volta che viene chiamata la funzione di schedulazione la coda viene analizzata alla ricerca del processo più vicino alla sua scadenza, il quale viene schedulato per l'esecuzione.

Particolarmente ottimo per i sistemi single-core con pre-rilascio.

Nel caso in cui ci siano processi periodici, anch'essi con delle scadenze (uguali al loro periodo) EDF ha un utilizzo della CPU vicino al 100%. Il test di schedulabilità per EDF è:

$$U = \sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

Limiti di questo algoritmo sono:

- Se il sistema è sovraccarico è impossibile prevedere il numero di processi che non rispettano la scadenza;
- Algoritmo difficile da implementare, per via di come dover rappresentare le scadenze

10.9 Sistemi Soft e Hard Real-Time

Un task real-time può essere distinto in 3 categorie:

- Hard → se la mancata scadenza può causare conseguenze catastrofiche sul sistema sotto controllo;
- Firm → se la produzione dei risultati dopo la scadenza è inutile per il sistema, ma non causa alcun danno;
- Soft → se la produzione dei risultati dopo la scadenza ha ancora qualche utilità per il sistema, sebbene causi un peggioramento delle prestazioni.

10.9.1 Gestione di task in contesti ibridi

Le applicazioni del mondo reale includon hard, firm e soft task, per cui un sistema real-time deve essere progettato per gestire i diversi tipi di task implementando strategie diverse:

- **Hard** → devono essere garantite offline;
- **Soft** → devono essere garantite online. interrompendoli se la loro scadenza non può essere rispettata;
- **Soft** → devono essere gestiti con l'obiettivo di ridurre al minimo il tempo di risposta medio.

10.9.2 Limiti dei comuni sistemi Real-Time

La gran parte dei sistemi real-time adottati per il controllo dei sistemi è basata su kernel, che sono riadattamenti degli analoghi utilizzati in sistemi time-sharing, con conseguenti svantaggi:

- Multitasking → gestita mediante system call, che non tengono esplicitamente considerazione del tempo, provocando ritardi imprevedibili;
- Scheduling basato su priorità → molto flessibili, ma hanno un numero limitato di livelli di priorità e non sempre i requisiti di tempo sono mappabili su tali intervalli. Inoltre, l'arrivo di un nuovo task potrebbe richiedere il ricalcolo di tutte le priorità;
- Reattività agli interrupt esterni → agli interrupt esterni è assegnata una priorità più alta di quella dei task da eseguire, che potrebbero rimanere sospesi per un tempo imprevedibile;
- IPC e sincronizzazione → i semafori binari normalmente usati provocando fenomeni quali inversione della priorità, attese a catena e deadlock, che rendono imprevedibile la durata dei task;
- Kernel ridotti e contest switch veloci → benchè questi riducano l'overhead del sistema non garantiscono esplicitamente il rispetto della deadline;
- Assenza di un real-time clock → non offrono la possibilità di gestire in modo esplicito il timing dei task.

10.9.3 Desiderata per i Sistemi Real-Time

Applicazioni di controllo complesse che richiedono forti vincoli temporali nell'esecuzione dei task devono essere supportati da sistemi operativi altamente prevedibili.

La prevedibilità può essere ottenuta solo introducendo cambiamenti radicali nei paradigmi di progettaione di base dei sistemi classici timesharing.

Il sistema è in grado di verificarne la schedulabilità, avendo come vincolo il rispetto delle deadline piuttosto che la riduzione del tempo medio di risposta.

Fra i desiderata per un SO real-time si annoverano:

- Timeliness → validità del risultato è intesa sia in termini di valore, che di tempo entro il quale esso è fornito;
- Prevedibilità → il SO deve essere in grado di prevedere le conseguenze di qualunque scelta nello scheduling;
- Efficienza → questo tipo di sistemi generalmente gira su HW limitati;
- Robustezza → non devono collassare in caso di eccessi di carico;
- Fault Tolerance → problemi HW o SW non devono mandare in crash l'intero sistema;
- Manutenibilità → devono essere progettati secondo uno schema modulare in modo da semplificare modifiche e integrazioni.

10.9.4 Prevedibilità

Il più importante tra i requisiti di un sistema hard real-time. Influenzata da fattori HW quali:

- DMA;
- Cache;
- Interrupts;
- System calls;
- Semaphores;
- Memory Management;
- Programming Language.

Gli Interrupts

Rappresentano un grosso problema per la prevedibilità di un sistema real-time, perchè, se non gestiti correttamente, possono introdurre ritardi notevoli nell'esecuzione di un task. Distinguiamo 3 approcci diversi:

- A → tutti gli interrupt esterni sono disabilitati a meno del timer
  - Le operazioni di `I/O` sono gestite direttamente dal processo mediante polling;
- B → tutti gli interrupt esterni sono disabilitati a meno del timer
  - Le operazioni di `I/O` sono gestite da routine del kernel attivate periodicamente dal timer;

- C → tutti gli interrupt esterni sono abilitati, ma i driver hanno un ruolo minimale
  - L'unica mansione di un driver è attivare un task il cui scopo è gestire il dispositivo;
  - La priorità di questi task è inferiore rispetto a quelli con deadline stringenti

I semafori

Questo meccanismo non è adatto a sistemi in tempo reale perchè soggetto al fenomeno di inversione di priorità, che si verifica quando un'attività ad alta priorità viene bloccata da un'attività a bassa priorità per un intervallo di tempo illimitato. Varie soluzioni efficaci a questo problema:

- Basic Priority Inheritance;
- Priority Ceiling;
- Stack Resource Policy.

10.10 I linguaggi di programmazione

Gli attuali linguaggi di programmazione non sono abbastanza espressivi da caratterizzare determinati comportamenti temporali e, quindi, non sono adatti per realizzare applicazioni prevedibili in tempo reale.

Sono stati proposti nuovi linguaggi ad alto livello per supportare lo sviluppo di applicazioni hard real-time:

- Real-Time Euclid;
- Real-Time Concurrent C.

10.10.1 Real-Time Euclid

Impone al programmatore di specificare limiti temporali ed eccezioni di timeout in tutti i loop, le attese e ele istruzioni di accesso ai dispositivi. Impone altre restrizioni quali:

- Assenza di strutture dinamiche → renderebbero imprevedibili i tempi di allocazione e deallocazione;
- Assenza di ricorsione → questa impedirebbe la stime del tempo di esecuzione delle chiamate ricorsive nell'analisi della schedulabilità;
- Cicli temporizzati → il programmatore deve specificare il numero massimo di iterazioni in ogni ciclo al fine di rendere possibilie la stima della durata di un task.

10.10.2 Real-Time Concurrent C

Estende il concurrent C fornendo delle facility per indicare la `periodicità/aperiodicità` e le deadline. Fornisce costrutti del tipo:

within deadline (d) statement-1

[else statement-2]

Costrutti quindi che permettono di specificare le istruzioni da eseguire nel caso in cui la deadline sia (non sia) rispettata

A differenza di Real-Time Euclid, che è stato progettato per supportare la verifica statica di sistemi real-time, Real-Time Concurrent C è orientato a sitemi dinamici, in cui è possibile attivare task in fase di esecuzione.

10.11 Sistemi Embedded

10.11.1 Caso Semplice I

Unico task periodico ripetuto ciclicamente.

All'accensione il sistema esegue direttamente l'algoritmo di controllo, scritto a partire dalla prima posizione in memoria (ROM) che consiste in un ciclo perpetuo che esegue il task.

Le deadline sono verificate se l'esecuzione del task è sufficientemente veloce

10.11.2 Caso Semplice II

Unico task periodico, invocato a intervalli regolari da un timer.

Il timer lancia un'interruzione, il task è realizzato dalla routine di servizio per l'interruzione relativa, la CPU altrimenti è in IDLE.

Le deadline sono verificate se l'esecuzione del task è sufficientemente veloce, mentre l'interruzione non interrompe la normale esecuzione.

10.11.3 Caso Complesso I

Più task periodici, di ugual periodo.

Ad ogni interruzione del timer vengono eseguiti tutti i task.

Le deadline sono verificate se la somma dei tempi di esecuzione dei task è inferiore alla deadline.

Se i periodi non sono uguali, ma multipli tra loro, le interruzioni seguono il periodo minore e ad ogni interruzione si sceglie quali task eseguire.

10.11.4 Caso Complesso II

Più task periodici, di periodo diverso, e task sporadici.

Le interruzioni dal timer arrivano secondo i vari periodo, e vi sono ache le interruzioni relative agli eventi che scatenano i task sporadici. Ogni interruzione è gestita da una propria routine che esegue il task relativo.

La verifica della deadline diventa difficile.

10.11.5 Arduino

Strumento open source che semplifica la progettazione e la prototipazione elettronica.

La scheda arduino presenta un HW molto modificabile, riprogettabile ed estensibili senza alcun limite (si sfruttano le librerie C++).

10.11.6 Microcontroller

Dispositivo elettronico integrato su un unico chip, progettato appositamente per interagire con input esterni, analogici o digitali, e restituire output analogici o digitali

10.11.7 La scheda Arduino

Ogni scheda presenta un ulteriore chip per la conversione del segnale digitale da USB a seriale.

Tramite il SW di Arduini è possibile programmare i pin come porte di entrata o di uscita digitale o analogica e se ne può controllare il comportamento

10.11.8 Programmare in Arduini

Comporta sostanzialmente scrivere due funzioni fondamentali:

- `setup();`
- `loop()`.

10.12 RT Linux

SO nel quale un piccolo kernel real-time coesiste con il kernel di Linux (like-POSIX).

L'intenzione è quella di fare uso dei servizi sofisticati ed altamente ottimizzati (per quel che concerne il comportamento medio) di un sistema di computer standard time-shared, mentre ancora si permette alle funzioni real-time di operare in un ambiente prevedibile ed a bassa latenza.

In questo SO un task real-time è eseguito su un kernel non real-time, che è concepito come un task a più bassa priorità, utilizzando un livello di VM per rendere il kernel standard completamente prelazionabile.

10.13 RTAI

Versione di Linux in cui sono state introdotte una serie di regole, che limitano il potenziale del Linux standard ad agire come un SO real-time.